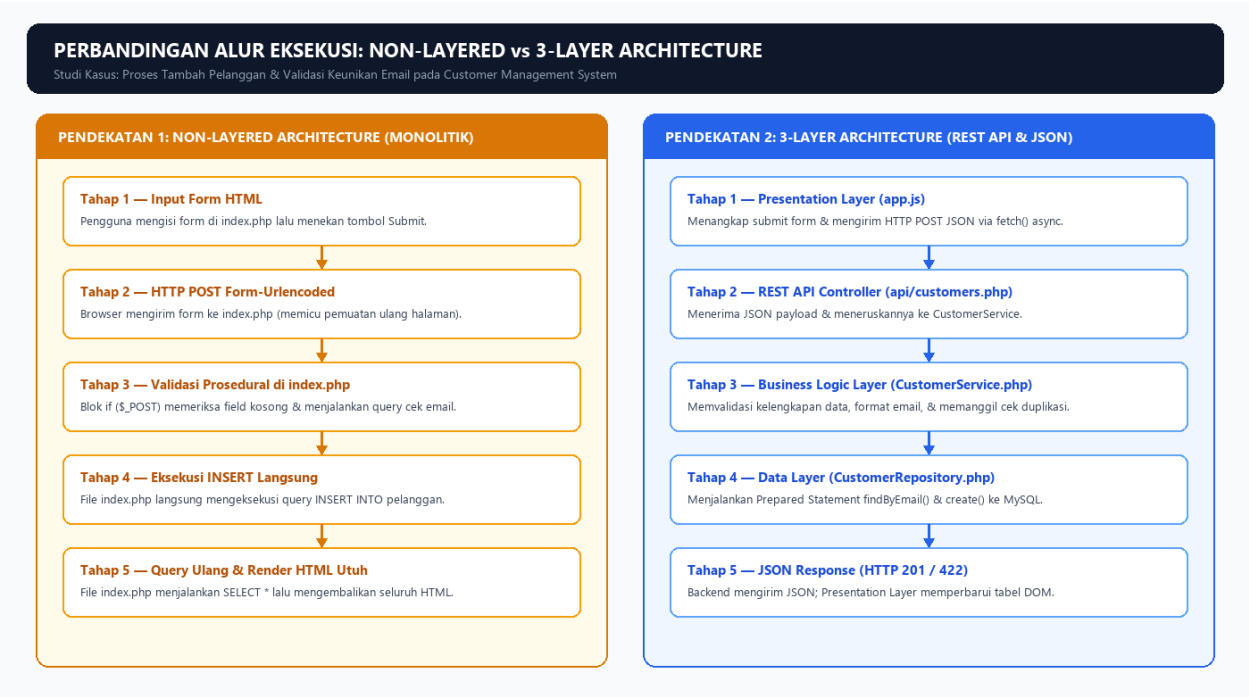
LAPORAN TUGAS KELOMPOK PEMROGRAMAN WEB

ANALISIS DAN IMPLEMENTASI PERBANDINGAN LAYERED VS NON-LAYERED WEB APPLICATION ARCHITECTURE

Studi Kasus: Customer Management System (PHP Native, REST API JSON, & MySQL)



Judul Tugas	Analisis dan Implementasi Perbandingan Layered vs Non-Layered Web Application Architecture
Studi Kasus	Customer Management System (Pengelolaan Data Pelanggan Perusahaan)
6 Fitur Wajib	1. Tambah Pelanggan, 2. Tampil Pelanggan, 3. Edit Pelanggan, 4. Hapus Pelanggan, 5. Pencarian (Nama/Email), 6. Validasi (Kelengkapan & Email Unik)
Aplikasi 1 (Non-Layered)	Tanpa Pemisahan Layer Tegas (bisa 1/beberapa file tercampur; pada studi kasus ini diimplementasikan dalam 1 file 1-non-layered-app/index.php)
Aplikasi 2 (3-Layer)	Pemisahan 3 Folder: presentation/ (HTML/CSS/JS), business-logic/ (Service & REST API JSON), data-access/ (PDO MySQL)
Database SQL	database/customer_management.sql (Database MySQL customer_management, Tabel pelanggan + 5 Data Awal)

BAB 1. PENDAHULUAN

1.1 Latar Belakang

Dalam pengembangan aplikasi berbasis web, terdapat berbagai pendekatan arsitektur yang dapat dipilih oleh pengembang untuk menyusun kode tampilan antarmuka (*frontend*), logika proses bisnis (*business logic*), serta pengelolaan database (*data access*). Dua pendekatan utama yang sering dipelajari dan dibandingkan dalam rekayasa perangkat lunak web adalah **Non-Layered Architecture** (arsitektur tanpa pemisahan layer secara tegas) dan **Layered Architecture** (khususnya **3-Layer Web Application Architecture**).

Pendekatan Pertama (Non-Layered Architecture) merupakan pendekatan di mana aplikasi dikembangkan tanpa pemisahan layer secara tegas; kode tampilan, proses bisnis, dan akses database **dapat berada dalam satu file atau beberapa file yang belum memiliki pemisahan tanggung jawab yang jelas** (pada implementasi studi kasus proyek ini ditempatkan dalam satu file index.php). Sementara itu, **Pendekatan Kedua (3-Layer Architecture)** memisahkan tanggung jawab aplikasi secara tegas ke dalam tiga lapisan yang berbeda—yaitu *Presentation Layer*, *Business Logic Layer*, dan *Data Layer*—yang saling berkomunikasi melalui protokol standar seperti REST API dan JSON. Kedua pendekatan ini masing-masing memiliki karakteristik, cara kerja, serta kesesuaian tersendiri tergantung pada skala dan kebutuhan proyek. Oleh karena itu, melalui studi kasus **Customer Management System** ini, kedua arsitektur diimplementasikan secara langsung untuk diuji dan dibandingkan secara objektif.

1.2 Rumusan Masalah

1. Apa pengertian, karakteristik, dan cara kerja dari **Non-Layered Architecture** maupun **3-Layer Web Application Architecture**?
2. Bagaimana cara mengimplementasikan studi kasus **Customer Management System** (dengan 6 fitur wajib) menggunakan pendekatan Non-Layered dibandingkan dengan pendekatan 3-Layer (3 folder terpisah dengan komunikasi REST API & JSON)?
3. Bagaimana perbedaan struktur pemisahan file/folder serta alur proses *request-response* pada kedua pendekatan arsitektur tersebut?
4. Berdasarkan hasil pengujian pada berbagai unsur (struktur file/folder, kecepatan pembuatan awal, pengerjaan individu vs tim, kemudahan penelusuran kesalahan/bug, kecepatan respons UI, keamanan edit desain, penambahan fitur, dan reusabilitas API), bagaimana perbandingan hasil kedua arsitektur dan kapan masing-masing lebih unggul?

1.3 Tujuan Penulisan & Implementasi

- Memahami konsep dasar, struktur, dan cara kerja **Non-Layered Architecture** serta **3-Layer Web Application Architecture** secara objektif.
- Membangun dua aplikasi web **Customer Management System** menggunakan PHP Native dan MySQL dengan 6 fitur wajib yang identik pada kedua arsitektur.
- Menyajikan perbandingan visual melalui **Diagram Arsitektur (Pemisahan File/Folder)**, **Screenshot Code** kedua konsep, serta **Screenshot Tampilan Web**.
- Menyusun **Satu Tabel Utama Hasil Pengujian dan Perbandingan** untuk menganalisis keunggulan dan alasan pemilihan masing-masing arsitektur secara objektif.

BAB 2. LANDASAN TEORI & KONSEP DASAR KEDUA ARSITEKTUR

2.1 Konsep Pendekatan Pertama: Non-Layered Architecture (Halaman 14–16 & 18 PDF)

Non-Layered Architecture (Aplikasi Tanpa 3-Layer) merupakan pendekatan pengembangan aplikasi web yang biasa disebut sebagai **arsitektur monolitik sederhana** (*monolithic application*), yaitu aplikasi yang dikembangkan **tanpa pemisahan layer secara tegas**. Sebagaimana dijelaskan pada Halaman 16 dan 18 PDF, kode tampilan (*frontend*), proses bisnis (validasi), dan akses database (query SQL) **dapat berada dalam satu**

file atau beberapa file yang belum memiliki pemisahan tanggung jawab (layer) yang jelas. Pada studi kasus sederhana (sesuai Halaman 14 & 20 PDF), seluruh proses tersebut ditempatkan secara terpadu di dalam **satu file PHP (index.php)** mulai dari: (1) Menampilkan formulir, (2) Menerima input pengguna, (3) Memvalidasi data, (4) Mengakses database MySQL, hingga (5) Menampilkan hasil.

Catatan Penting (Halaman 16 PDF): *Aplikasi tanpa 3-layer ini bukan berarti sama sekali tidak memiliki tiga tanggung jawab tersebut. Fungsi tampilan, proses bisnis, dan akses data tetap ada di dalamnya, tetapi tidak dipisahkan menjadi layer yang jelas.*

2.2 Konsep Pendekatan Kedua: 3-Layer Web Application Architecture (Halaman 2–11 PDF)

3-Layer Web Application Architecture adalah konsep pengembangan aplikasi web dengan membagi sistem menjadi tiga lapisan (*layer*) yang memiliki tugas dan tanggung jawab berbeda:

1. **Presentation Layer (Lapisan Antarmuka / Frontend):** Bagian aplikasi yang berinteraksi langsung dengan pengguna, bertugas menampilkan informasi dan menerima masukan dari pengguna menggunakan **HTML**, **CSS**, dan **JavaScript**. *Intinya: Presentation Layer bertanggung jawab terhadap apa yang dilihat dan digunakan oleh pengguna.*
2. **Application / Business Logic Layer (Lapisan Logika Bisnis / Backend):** Bagian yang menangani proses bisnis atau aturan utama aplikasi, menerima permintaan dari frontend, memproses/memvalidasinya, kemudian berkomunikasi dengan database jika diperlukan. *Intinya: Business Logic Layer bertanggung jawab terhadap bagaimana aplikasi bekerja dan mengambil keputusan.*
3. **Data Layer (Lapisan Pengelolaan Data / Database):** Bagian yang bertugas menyimpan, mengambil, memperbarui, dan menghapus data di dalam database (seperti **MySQL**). *Intinya: Data Layer bertanggung jawab terhadap penyimpanan dan pengelolaan data aplikasi.*

Aspek	Presentation Layer	Business Logic Layer	Data Layer
Fungsi Utama	Tampilan & Interaksi User	Proses & Aturan Bisnis	Penyimpanan & Query Data
Berinteraksi Dengan	Pengguna (Browser)	Frontend & Data Layer	Business Logic Layer (Backend)
Teknologi Proyek Ini	HTML5, CSS3, JavaScript (Fetch)	PHP Native (Service & REST API)	PDO PHP & Database MySQL
Folder pada Proyek	2-layered-app/presentation/	2-layered-app/business-logic/	2-layered-app/data-access/
Tanggung Jawab	Menampilkan & menerima data	Memvalidasi & mengolah data	Menyimpan & mengambil data SQL

Tabel 2.1 Pembagian Tanggung Jawab Ketiga Layer pada Arsitektur 3-Layer (Sesuai Halaman 11 PDF)

2.3 Karakteristik Arsitektur Non-Layered vs 3-Layer (Halaman 12–16 PDF)

Kedua pendekatan arsitektur memiliki cara kerja dan karakteristik struktural yang berbeda. Pada **Non-Layered Architecture** (Halaman 15 PDF), alur kerja berjalan dalam 6 tahap langsung di satu file: (1) Pengguna mengisi formulir, (2) Browser mengirim data ke index.php via HTTP POST, (3) PHP memvalidasi dan menjalankan query SQL, (4) MySQL menyimpan data, (5) PHP menjalankan SELECT data terbaru, dan (6) Browser menampilkan kembali halaman HTML utuh.

Sementara itu, pada **3-Layer Architecture** (Halaman 10 & 12 PDF), sistem dirancang berdasarkan lima karakteristik arsitektural: (1) **Separation of Concerns** (pemisahan tanggung jawab tiap layer), (2) **Maintainability** (kemudahan pemeliharaan tampilan & database secara terpisah), (3) **Scalability** (pengembangan tiap layer sesuai kebutuhan), (4) **Security** (isolasi akses database di Data Layer), serta (5) **Reusability** (penggunaan ulang API oleh berbagai platform). Perbandingan karakteristik dasar kedua konsep tersebut dirangkum pada Tabel 2.2:

Aspek Karakteristik	Pendekatan 1: Tanpa 3-Layer (Non-Layered)	Pendekatan 2: Dengan 3-Layer Architecture
Struktur Kode	Sederhana (belum memiliki pemisahan layer yang tegas)	Terpisah secara tegas ke dalam 3 layer
File Program	Bisa satu file atau beberapa file tanpa batas layer (di sini 1 file index.php)	Dibagi ke beberapa komponen/folder sesuai layer
Bagian Frontend	Menyatu langsung dengan kode PHP	Terpisah di folder presentation/
Bagian Business Logic	Menyatu dengan kode aplikasi & tampilan	Dipisahkan di folder business-logic/
Bagian Database Access	Langsung ditulis di dalam file PHP utama	Melalui komponen khusus di folder data-access/
Karakteristik Pemeliharaan	Praktis saat kecil, lebih rumit jika fitur sangat kompleks	Lebih terstruktur karena perubahan terlokalisasi per layer
Karakteristik Pengembangan	Sangat cepat untuk pembuatan aplikasi kecil / individu	Lebih terorganisasi untuk pengembangan tim & jangka panjang
Karakteristik Pengujian	Diuji secara keseluruhan dalam satu file terpadu	Dapat diuji secara terpisah pada masing-masing layer

Tabel 2.2 Perbandingan Karakteristik Dasar Tanpa 3-Layer vs Dengan 3-Layer (Sesuai Halaman 16 PDF)

BAB 3. STUDI KASUS: CUSTOMER MANAGEMENT SYSTEM & DATABASE

3.1 Spesifikasi 6 Fitur Wajib (Halaman 18–19 PDF)

Sesuai instruksi pada Halaman 18–19 PDF, studi kasus yang dibangun adalah **Customer Management System**, yaitu aplikasi web untuk mengelola data pelanggan sebuah perusahaan. Kedua aplikasi (baik Non-Layered maupun 3-Layer) wajib mengimplementasikan **6 fitur utama** berikut:

No	Nama Fitur Wajib	Keterangan Fungsi	Implementasi pada Proyek
1	**Tambah Pelanggan**	Memasukkan data pelanggan baru	Input nama, email, telepon, dan alamat ke tabel pelanggan
2	**Tampil Pelanggan**	Menampilkan seluruh data pelanggan	Menampilkan tabel daftar pelanggan diurutkan dari ID terbaru
3	**Edit Pelanggan**	Mengubah informasi pelanggan	Mengambil data berdasarkan ID lalu memperbarui baris di MySQL
4	**Hapus Pelanggan**	Menghapus data pelanggan	Menghapus data pelanggan berdasarkan ID dengan konfirmasi
5	**Pencarian**	Mencari berdasarkan nama atau email	Query pencarian menggunakan WHERE nama LIKE ? OR email LIKE ?
6	**Validasi**	Memeriksa kelengkapan & keunikan email	Menolak kolom kosong, format email salah, atau email yang sudah terdaftar

Tabel 3.1 Enam Fitur Wajib Studi Kasus Customer Management System (Sesuai Halaman 19 PDF)

3.2 Struktur Tabel Database MySQL (File: database/customer_management.sql)

Kedua aplikasi menggunakan database MySQL bernama customer_management dengan tabel pelanggan yang sudah dilengkapi 5 data pelanggan awal (*seed data*):

Nama Kolom	Tipe Data	Constraint / Indeks	Keterangan
id	INT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT	ID unik setiap pelanggan
nama	VARCHAR(100)	NOT NULL, INDEX	Nama lengkap pelanggan perusahaan
email	VARCHAR(100)	NOT NULL, UNIQUE	Alamat email pelanggan (wajib unik / tidak boleh sama)
telepon	VARCHAR(25)	NOT NULL	Nomor telepon aktif pelanggan
alamat	TEXT	NOT NULL	Alamat lengkap pelanggan
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Waktu pencatatan data pelanggan

Tabel 3.2 Struktur Tabel pelanggan pada File database/customer_management.sql

3.3 Luaran 3: Seluruh Isi File Database SQL — Lokasi File: database/customer_management.sql (Total: 28 Baris Lengkap)

Berikut adalah seluruh kode SQL pada file database/customer_management.sql untuk membuat database customer_management, tabel pelanggan, beserta 5 data pelanggan awal (*seed data*):

Source Code Lengkap: database/customer_management.sql (Baris 1-28)

```sql
-- ============================================================================
-- FILE DATABASE SQL: Customer Management System
-- Tugas Kelompok : Perbandingan Layered vs Non-Layered Web Architecture
-- Cara Import    : Buka phpMyAdmin -> Buat/Pilih DB -> Menu Import -> Pilih file ini
-- ============================================================================

CREATE DATABASE IF NOT EXISTS customer_management;
USE customer_management;

-- Membuat tabel pelanggan (sesuai studi kasus)
DROP TABLE IF EXISTS pelanggan;

CREATE TABLE pelanggan (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nama VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    telepon VARCHAR(25) NOT NULL,
    alamat TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

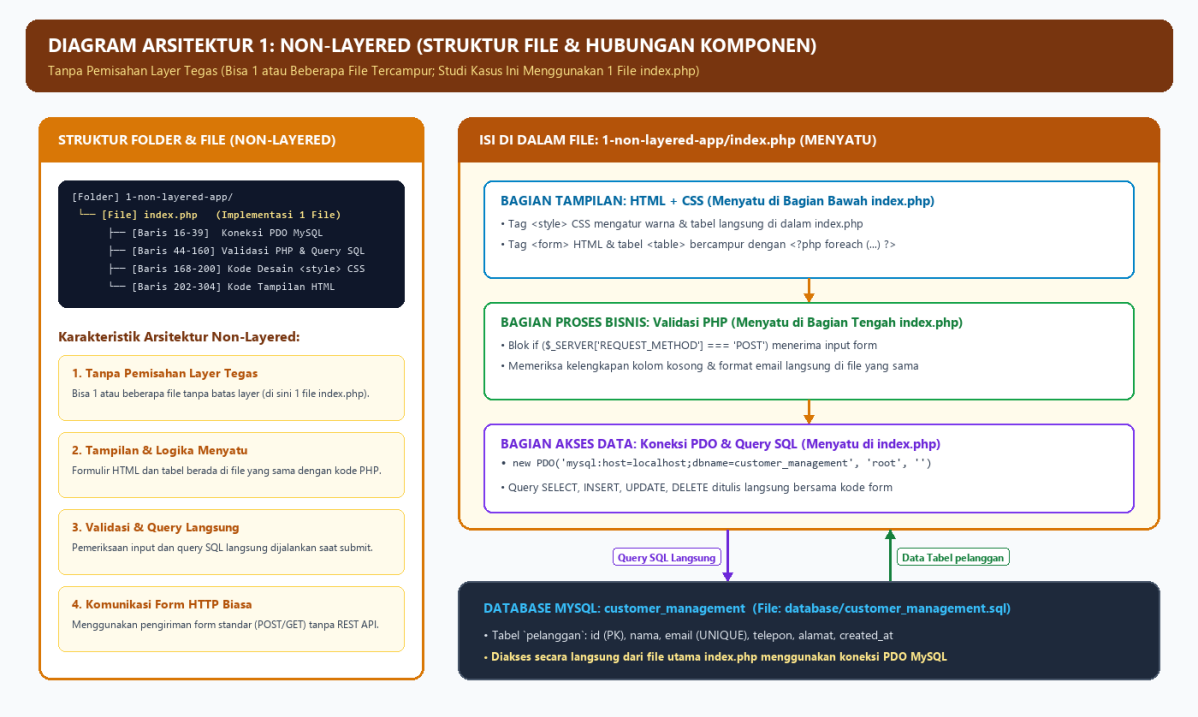
-- Memasukkan 5 data contoh pelanggan awal
INSERT INTO pelanggan (nama, email, telepon, alamat) VALUES
('Andi Pratama', 'andi@nusantara.co.id', '0812-3456-7890', 'Jl. Setiabudhi No. 229, Kota Bandung'),
('Budi Santoso', 'budi@teknologimaju.id', '0813-9876-5432', 'Jl. Asia Afrika No. 88, Kota Bandung'),
('Citra Lestari', 'citra@kreasidigital.com', '0821-5566-7788', 'Jl. Ir. H. Juanda (Dago) No. 145, Kota B
  andung'),
('Dewi Anggraini', 'dewi@sinergiglobal.co.id', '0852-1122-3344', 'Jl. Soekarno Hatta No. 512, Kota Bandu
  ng'),
('Eko Prasetyo', 'eko@mitraniaga.id', '0878-4455-6677', 'Jl. Pasteur No. 32, Kota Bandung');
```

3.4 Ketentuan Teknis Kedua Aplikasi (Halaman 20–21 PDF)

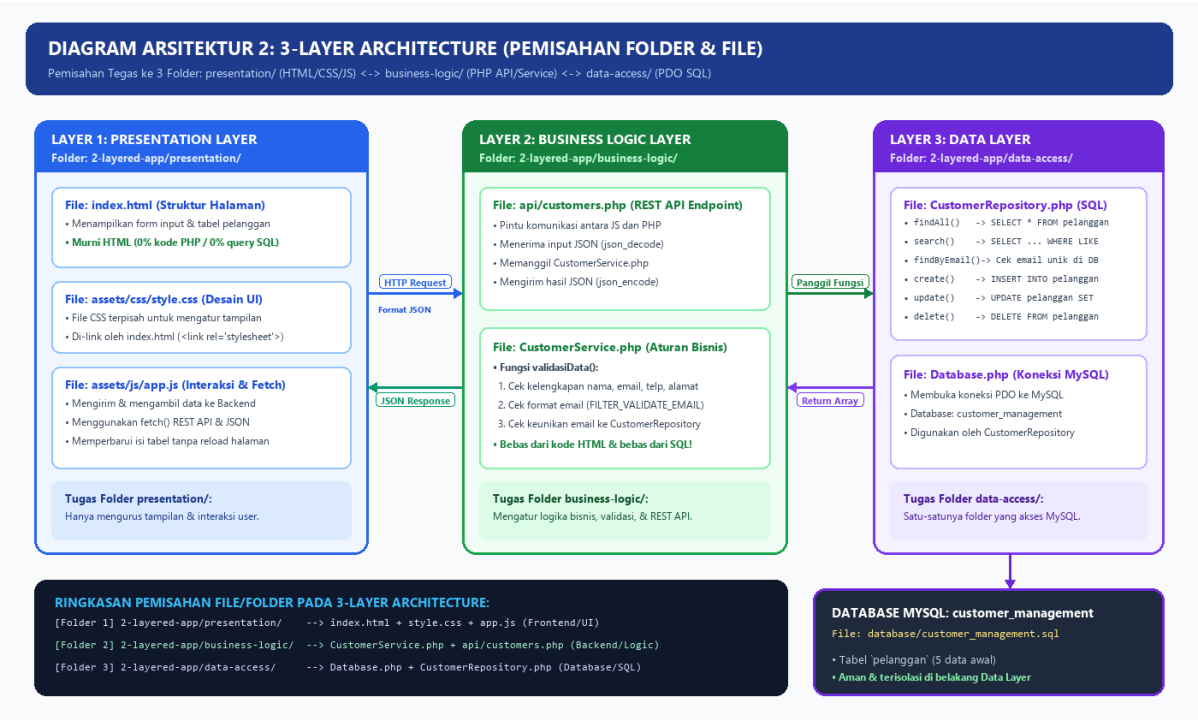
- **Aplikasi Pertama (Non-Layered Architecture — Halaman 20 PDF):** Dibangun menggunakan PHP Native dan MySQL di mana tampilan HTML digabungkan dengan PHP, proses bisnis ditempatkan dalam file yang sama dengan tampilan, koneksi database dan query ditulis langsung dalam file aplikasi (1-non-layered-app/index.php), tidak menggunakan REST API, dan seluruh fitur CRUD berjalan.
- **Aplikasi Kedua (3-Layer Architecture — Halaman 21 PDF):** Membangun kembali aplikasi yang sama menggunakan PHP Native (non-framework) dengan memisahkan tanggung jawab menjadi 3 folder: **Presentation Layer** (presentation/) menangani tampilan & interaksi user, **Business Logic Layer** (business-logic/) menangani proses & aturan bisnis, serta **Data Layer** (data-access/) menangani komunikasi dengan database MySQL. Komunikasi antara frontend dan backend wajib menggunakan **REST API dan JSON**.

BAB 4. LUARAN 4: DIAGRAM ARSITEKTUR (PEMISAHAN FILE/FOLDER & HUBUNGAN KOMPONEN)

Sesuai arahan dosen, diagram arsitektur dirancang untuk memperlihatkan secara jelas **struktur pemisahan file/folder** serta bagaimana komponen **HTML, CSS, JavaScript, PHP, dan SQL** saling berhubungan pada kedua pendekatan:



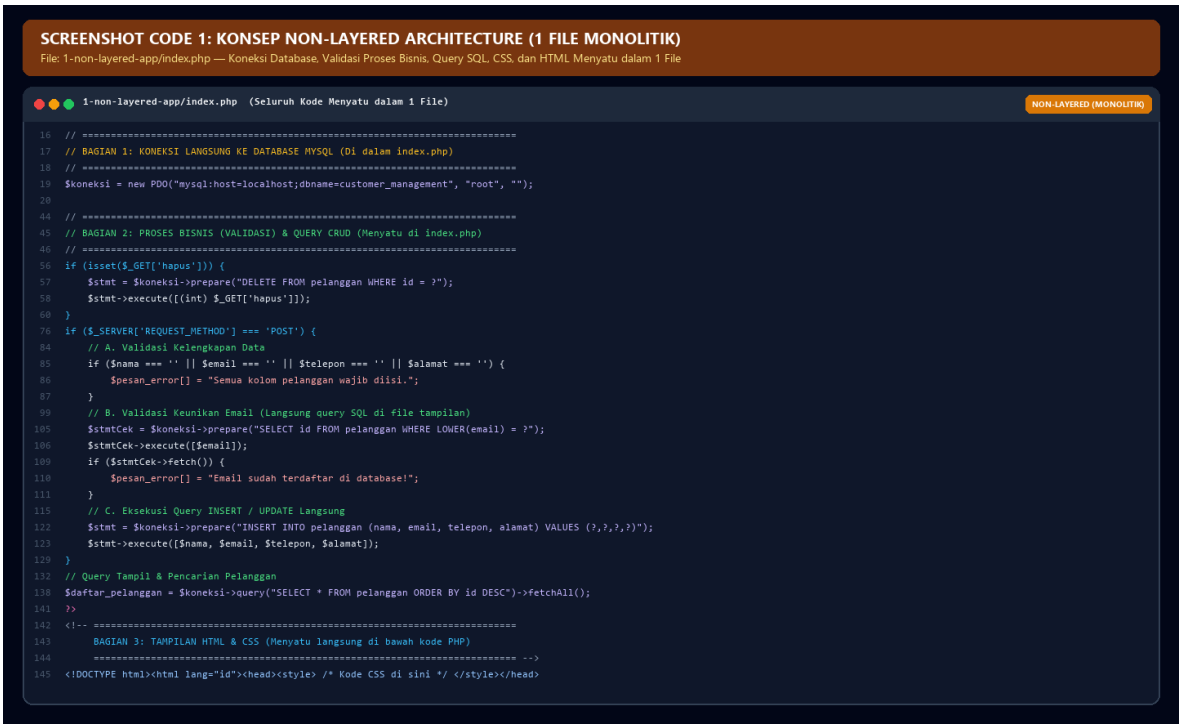
Gambar 4.1 Diagram Arsitektur Non-Layered: Tanpa Pemisahan Layer Tegas (Studi Kasus Ini Menggunakan 1 File index.php)



Gambar 4.2 Diagram Arsitektur 3-Layer: Pemisahan Tegas ke dalam 3 Folder Layer (presentation/, business-logic/, data-access/)

BAB 5. LUARAN 1 & 2: SCREENSHOT CODE & SELURUH SOURCE CODE KEDUA APLIKASI

Untuk membuktikan perbedaan konsep secara nyata pada level kode sumber (*source code*), bab ini menyajikan **Screenshot Code** yang mewakili kedua konsep arsitektur, diikuti dengan **seluruh baris kode sumber lengkap (100%)** dari Aplikasi Pertama (Non-Layered) dan Aplikasi Kedua (3-Layer Architecture):



Gambar 5.1 Screenshot Code Non-Layered Architecture (1-non-layered-app/index.php): Koneksi PDO, Validasi, Query SQL, CSS, dan HTML Menyatu



Gambar 5.2 Screenshot Code 3-Layer Architecture: Pemisahan Tegas antara app.js (Presentation), CustomerService.php (Logic), dan CustomerRepository.php (Data)

5.1 Luaran 1: Seluruh Source Code Aplikasi Non-Layered Architecture

Aplikasi 1 (Non-Layered Architecture) — Lokasi File: 1-non-layered-app/index.php (Total: 310 Baris Lengkap)

Pada file tunggal 1-non-layered-app/index.php (310 baris) di bawah ini, seluruh tanggung jawab aplikasi mulai dari koneksi PDO MySQL & *auto-seed database* (Baris 16–39), proses bisnis validasi kelengkapan & keunikan email (Baris 44–100), eksekusi query SQL INSERT, UPDATE, DELETE, dan SELECT pencarian (Baris 101–160), desain <style> CSS (Baris 168–200), hingga struktur <form> dan <table> HTML (Baris 202–310) ditempatkan menyatu di dalam satu file yang sama:

Source Code Lengkap: 1-non-layered-app/index.php (Bagian 1/7)

```
001 | <?php
002 | /**
003 |  * ============================================================================
004 |  * APLIKASI 1: NON-LAYERED ARCHITECTURE (ARSITEKTUR MONOLITIK 1 FILE)
005 |  * Studi Kasus : Customer Management System
006 |  * Sesuai PDF  : Halaman 14, 15, 19, dan 20
007 |  *
008 |  * Ciri Utama  :
009 |  * 1. Koneksi database MySQL dan query SQL ditulis langsung di dalam file ini.
010 |  * 2. Proses bisnis (validasi kelengkapan & keunikan email) ada di file ini.
011 |  * 3. Tampilan HTML digabungkan langsung dengan kode PHP di file ini.
012 |  * 4. Tidak menggunakan REST API (menggunakan submit form biasa).
013 |  * ============================================================================
014 |  */
015 |
016 | // ============================================================================
017 | // BAGIAN 1: KONEKSI LANGSUNG KE DATABASE MYSQL
018 | // ============================================================================
019 | $host     = "localhost";
020 | $dbname   = "customer_management";
021 | $username = "root";
022 | $password = "";
023 |
024 | $options = [
025 |     PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
026 |     PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
027 | ];
028 |
029 | try {
030 |     $koneksi = new PDO("mysql:host=$host;dbname=$dbname;charset=utf8mb4", $username, $password, $options
    |   );
031 | } catch (PDOException $e) {
032 |     // Jika database customer_management belum di-import di MySQL, otomatis jalankan file database/custo
    |   mer_management.sql
033 |     $sqlFile = dirname(__DIR__) . "/database/customer_management.sql";
034 |     if (strpos($e->getMessage(), "1049") !== false && file_exists($sqlFile)) {
035 |         $pdoSetup = new PDO("mysql:host=$host;charset=utf8mb4", $username, $password, $options);
036 |         $pdoSetup->exec(file_get_contents($sqlFile));
037 |         $koneksi = new PDO("mysql:host=$host;dbname=$dbname;charset=utf8mb4", $username, $password, $opt
    |   ions);
038 |     } else {
039 |         die("Koneksi MySQL gagal. Pastikan MySQL di XAMPP/Laragon sudah dinyalakan. Detail: " . $e->getM
    |   essage());
040 |     }
041 | }
042 |
043 | // ============================================================================
044 | // BAGIAN 2: PROSES BISNIS (VALIDASI) & QUERY CRUD KE DATABASE MYSQL
045 | // ============================================================================
046 | $pesan_error  = [];
047 | $pesan_sukses = "";
048 |
```

Source Code Lengkap: 1-non-layered-app/index.php (Bagian 2/7)

```
049 | // Variabel penampung nilai form (untuk Tambah maupun Edit)
050 | $id_edit = "";
051 | $nama    = "";
052 | $email   = "";
053 | $telepon = "";
054 | $alamat  = "";
055 |
056 | // --- FITUR 4: HAPUS PELANGGAN (Saat tombol Hapus diklik) ---
057 | if (isset($_GET['hapus'])) {
058 |     $id_hapus = (int) $_GET['hapus'];
059 |     $stmt = $koneksi->prepare("DELETE FROM pelanggan WHERE id = ?");
060 |     $stmt->execute([$id_hapus]);
061 |     $pesan_sukses = "Data pelanggan berhasil dihapus dari database.";
062 | }
063 |
064 | // --- PERSIAPAN FITUR 3: AMBIL DATA DARI DATABASE UNTUK FORM EDIT ---
065 | if (isset($_GET['edit'])) {
066 |     $id_ambil = (int) $_GET['edit'];
067 |     $stmt = $koneksi->prepare("SELECT * FROM pelanggan WHERE id = ?");
068 |     $stmt->execute([$id_ambil]);
069 |     $data_lama = $stmt->fetch();
070 |
071 |     if ($data_lama) {
072 |         $id_edit = $data_lama['id'];
073 |         $nama    = $data_lama['nama'];
074 |         $email   = $data_lama['email'];
075 |         $telepon = $data_lama['telepon'];
076 |         $alamat  = $data_lama['alamat'];
077 |     }
078 | }
079 |
080 | // --- FITUR 1, 3, & 6: SIMPAN / UPDATE PELANGGAN & VALIDASI DATA ---
081 | if ($_SERVER['REQUEST_METHOD'] === 'POST') {
082 |     $id_edit = trim($_POST['id_edit'] ?? '');
083 |     $nama    = trim($_POST['nama'] ?? '');
084 |     $email   = strtolower(trim($_POST['email'] ?? ''));
085 |     $telepon = trim($_POST['telepon'] ?? '');
086 |     $alamat  = trim($_POST['alamat'] ?? '');
087 |
088 |     // A. Validasi Kelengkapan Data (Semua kolom wajib diisi)
089 |     if ($nama === '') {
090 |         $pesan_error[] = "Nama pelanggan wajib diisi.";
091 |     }
092 |     if ($email === '') {
093 |         $pesan_error[] = "Email pelanggan wajib diisi.";
094 |     } elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
095 |         $pesan_error[] = "Format alamat email tidak valid.";
096 |     }
097 |     if ($telepon === '') {
098 |         $pesan_error[] = "Nomor telepon wajib diisi.";
099 |     }
100 |     if ($alamat === '') {
```

Source Code Lengkap: 1-non-layered-app/index.php (Bagian 3/7)

```
        $pesan_error[] = "Alamat pelanggan wajib diisi.";
    }

    // B. Validasi Keunikan Email (Cek langsung ke tabel pelanggan di MySQL)
    if (empty($pesan_error)) {
        if ($id_edit !== '') {
            // Saat Edit: cek apakah email dipakai oleh pelanggan selain dirinya sendiri
            $stmtCek = $koneksi->prepare("SELECT id FROM pelanggan WHERE LOWER(email) = ? AND id != ?");
            $stmtCek->execute([$email, (int) $id_edit]);
        } else {
            // Saat Tambah Baru: cek apakah email sudah ada di tabel pelanggan
            $stmtCek = $koneksi->prepare("SELECT id FROM pelanggan WHERE LOWER(email) = ?");
            $stmtCek->execute([$email]);
        }

        if ($stmtCek->fetch()) {
            $pesan_error[] = "Email '$email' sudah terdaftar di database. Gunakan email yang unik!";
        }
    }

    // C. Jika Validasi Lolos -> Jalankan Query INSERT atau UPDATE ke MySQL
    if (empty($pesan_error)) {
        if ($id_edit !== '') {
            // FITUR 3: EDIT PELANGGAN (UPDATE)
            $stmt = $koneksi->prepare("UPDATE pelanggan SET nama = ?, email = ?, telepon = ?, alamat = ?
   WHERE id = ?");
            $stmt->execute([$nama, $email, $telepon, $alamat, (int) $id_edit]);
            $pesan_sukses = "Data pelanggan '$nama' berhasil diperbarui!";
        } else {
            // FITUR 1: TAMBAH PELANGGAN BARU (INSERT)
            $stmt = $koneksi->prepare("INSERT INTO pelanggan (nama, email, telepon, alamat) VALUES (?, ?
  , ?, ?)");
            $stmt->execute([$nama, $email, $telepon, $alamat]);
            $pesan_sukses = "Pelanggan baru '$nama' berhasil ditambahkan ke database!";
        }

        // Kosongkan form kembali setelah berhasil disimpan
        $id_edit = $nama = $email = $telepon = $alamat = "";
    }
}

// --- FITUR 2 & 5: TAMPIL PELANGGAN & PENCARIAN DARI DATABASE MYSQL ---
$kata_kunci = trim($_GET['cari'] ?? '');
if ($kata_kunci !== '') {
    $stmt = $koneksi->prepare("SELECT * FROM pelanggan WHERE nama LIKE ? OR email LIKE ? ORDER BY id DES
  C");
    $stmt->execute(["%$kata_kunci%", "%$kata_kunci%"]);
    $daftar_pelanggan = $stmt->fetchAll();
} else {
    $stmt = $koneksi->query("SELECT * FROM pelanggan ORDER BY id DESC");
    $daftar_pelanggan = $stmt->fetchAll();
}
```

Source Code Lengkap: 1-non-layered-app/index.php (Bagian 4/7)

```
150 | ?>
151 | <!-- ========================================================================
152 |      BAGIAN 3: TAMPILAN HTML & CSS (Menyatu langsung dengan kode PHP di atas)
153 |      ======================================================================== -->
154 | <!DOCTYPE html>
155 | <html lang="id">
156 | <head>
157 |     <meta charset="UTF-8">
158 |     <title>Customer Management System - Non-Layered Architecture</title>
159 |     <style>
160 |         * { box-sizing: border-box; font-family: 'Segoe UI', Arial, sans-serif; }
161 |         body { background: #f4f6f9; margin: 0; padding: 0; color: #1e293b; }
162 |         .header { background: #b45309; color: white; padding: 18px 30px; }
163 |         .header h1 { margin: 0; font-size: 22px; }
164 |         .header p { margin: 4px 0 0; font-size: 14px; opacity: 0.9; }
165 |         .container { max-width: 1150px; margin: 20px auto; padding: 0 20px; }
166 |         .info-box { background: #fffbeb; border-left: 5px solid #d97706; padding: 12px 16px; margin-bott
    |   om: 18px; border-radius: 6px; font-size: 14px; }
167 |         .alert-error { background: #fef2f2; border-left: 5px solid #dc2626; color: #991b1b; padding: 12p
    |   x 16px; margin-bottom: 18px; border-radius: 6px; }
168 |         .alert-success { background: #f0fdf4; border-left: 5px solid #16a34a; color: #166534; padding: 1
    |   2px 16px; margin-bottom: 18px; border-radius: 6px; font-weight: bold; }
169 |         .grid { display: grid; grid-template-columns: 350px 1fr; gap: 20px; align-items: start; }
170 |         @media (max-width: 850px) { .grid { grid-template-columns: 1fr; } }
171 |         .card { background: white; border: 1px solid #cbd5e1; border-radius: 8px; padding: 18px; box-sha
    |   dow: 0 1px 3px rgba(0,0,0,0.05); }
172 |         .card h2 { margin-top: 0; font-size: 18px; border-bottom: 1px solid #e2e8f0; padding-bottom: 10p
    |   x; }
173 |         .form-group { margin-bottom: 12px; }
174 |         .form-group label { display: block; font-weight: 600; font-size: 13px; margin-bottom: 5px; }
175 |         .form-group input, .form-group textarea { width: 100%; padding: 8px 10px; border: 1px solid #cbd
    |   5e1; border-radius: 5px; font-size: 14px; }
176 |         .btn { display: inline-block; padding: 8px 14px; border: none; border-radius: 5px; cursor: point
    |   er; font-weight: 600; font-size: 13px; text-decoration: none; }
177 |         .btn-utama { background: #d97706; color: white; }
178 |         .btn-utama:hover { background: #b45309; }
179 |         .btn-batal { background: #e2e8f0; color: #334155; }
180 |         .btn-edit { background: #dbeafe; color: #1d4ed8; padding: 5px 10px; }
181 |         .btn-hapus { background: #fee2e2; color: #dc2626; padding: 5px 10px; }
182 |         .search-form { display: flex; gap: 8px; margin-bottom: 14px; }
183 |         .search-form input { flex: 1; padding: 8px 10px; border: 1px solid #cbd5e1; border-radius: 5px;
    |   }
184 |         table { width: 100%; border-collapse: collapse; font-size: 14px; }
185 |         th, td { border-bottom: 1px solid #e2e8f0; padding: 10px; text-align: left; vertical-align: top;
    |    }
186 |         th { background: #f8fafc; font-size: 12px; text-transform: uppercase; color: #475569; }
187 |     </style>
188 | </head>
189 | <body>
190 |
191 | <div class="header">
192 |     <h1>Customer Management System — Aplikasi 1: Non-Layered Architecture</h1>
```

Source Code Lengkap: 1-non-layered-app/index.php (Bagian 5/7)

```
193 |     <p>Pendekatan Monolitik Sederhana: HTML, Validasi Proses Bisnis, dan Query Database Menyatu dalam 1
    |   File (index.php)</p>
194 | </div>
195 |
196 | <div class="container">
197 |     <div class="info-box">
198 |         <strong>Info Arsitektur:</strong> Pada aplikasi ini, seluruh proses mulai dari menampilkan form,
    |    menerima input, memvalidasi data, mengakses database MySQL, hingga menampilkan tabel pelanggan bera
    |   da di dalam satu file <code>index.php</code>.
199 |     </div>
200 |
201 |     <!-- Menampilkan Pesan Error Validasi (Jika Ada) -->
202 |     <?php if (!empty($pesan_error)): ?>
203 |         <div class="alert-error">
204 |             <strong>Validasi Gagal:</strong>
205 |             <ul style="margin: 6px 0 0; padding-left: 20px;">
206 |                 <?php foreach ($pesan_error as $err): ?>
207 |                     <li><?= htmlspecialchars($err) ?></li>
208 |                 <?php endforeach; ?>
209 |             </ul>
210 |         </div>
211 |     <?php endif; ?>
212 |
213 |     <!-- Menampilkan Pesan Sukses (Jika Ada) -->
214 |     <?php if ($pesan_sukses !== ""): ?>
215 |         <div class="alert-success">
216 |             &#10004; <?= htmlspecialchars($pesan_sukses) ?>
217 |         </div>
218 |     <?php endif; ?>
219 |
220 |     <div class="grid">
221 |         <!-- KOLOM KIRI: FORM TAMBAH / EDIT PELANGGAN -->
222 |         <div class="card">
223 |             <h2><?= $id_edit !== '' ? "Edit Pelanggan #$id_edit" : "Tambah Pelanggan Baru" ?></h2>
224 |             <form method="POST" action="index.php">
225 |                 <input type="hidden" name="id_edit" value="<?= htmlspecialchars((string)$id_edit) ?>">
226 |
227 |                 <div class="form-group">
228 |                     <label>Nama Pelanggan *</label>
229 |                     <input type="text" name="nama" value="<?= htmlspecialchars($nama) ?>" placeholder="C
    |   ontoh: Andi Pratama">
230 |                 </div>
231 |
232 |                 <div class="form-group">
233 |                     <label>Email Pelanggan (Unik) *</label>
234 |                     <input type="text" name="email" value="<?= htmlspecialchars($email) ?>" placeholder=
    |   "Contoh: andi@email.com">
235 |                 </div>
236 |
237 |                 <div class="form-group">
238 |                     <label>Nomor Telepon *</label>
239 |                     <input type="text" name="telepon" value="<?= htmlspecialchars($telepon) ?>" placehol
```

Source Code Lengkap: 1-non-layered-app/index.php (Bagian 6/7)

```
    |   der="Contoh: 0812-3456-7890">
240 |                 </div>
241 |
242 |                 <div class="form-group">
243 |                     <label>Alamat Lengkap *</label>
244 |                     <textarea name="alamat" rows="3" placeholder="Masukkan alamat lengkap..."><?= htmlsp
    |   ecialchars($alamat) ?></textarea>
245 |                 </div>
246 |
247 |                 <button type="submit" class="btn btn-utama">
248 |                     <?= $id_edit !== '' ? "Simpan Perubahan" : "+ Tambah Pelanggan" ?>
249 |                 </button>
250 |                 <?php if ($id_edit !== ''): ?>
251 |                     <a href="index.php" class="btn btn-batal">Batal</a>
252 |                 <?php endif; ?>
253 |             </form>
254 |         </div>
255 |
256 |         <!-- KOLOM KANAN: PENCARIAN & TABEL DATA PELANGGAN -->
257 |         <div class="card">
258 |             <h2>Daftar Pelanggan (Total: <?= count($daftar_pelanggan) ?>)</h2>
259 |
260 |             <!-- Form Pencarian Berdasarkan Nama atau Email -->
261 |             <form method="GET" action="index.php" class="search-form">
262 |                 <input type="text" name="cari" value="<?= htmlspecialchars($kata_kunci) ?>" placeholder=
    |   "Cari berdasarkan nama atau email...">
263 |                 <button type="submit" class="btn btn-utama">Cari</button>
264 |                 <?php if ($kata_kunci !== ''): ?>
265 |                     <a href="index.php" class="btn btn-batal">Reset</a>
266 |                 <?php endif; ?>
267 |             </form>
268 |
269 |             <table>
270 |                 <thead>
271 |                     <tr>
272 |                         <th>ID</th>
273 |                         <th>Nama &amp;amp; Email</th>
274 |                         <th>Telepon</th>
275 |                         <th>Alamat</th>
276 |                         <th style="width: 120px;">Aksi</th>
277 |                     </tr>
278 |                 </thead>
279 |                 <tbody>
280 |                     <?php if (empty($daftar_pelanggan)): ?>
281 |                         <tr>
282 |                             <td colspan="5" style="text-align: center; padding: 20px; color: #64748b;">
283 |                                 Data pelanggan tidak ditemukan.
284 |                             </td>
285 |                         </tr>
286 |                     <?php else: ?>
287 |                         <?php foreach ($daftar_pelanggan as $row): ?>
288 |                             <tr>
```

Source Code Lengkap: 1-non-layered-app/index.php (Bagian 7/7)

```
289 |                                 <td><strong>#<?= $row['id'] ?></strong></td>
290 |                                 <td>
291 |                                     <strong><?= htmlspecialchars($row['nama']) ?></strong><br>
292 |                                     <small style="color: #0369a1;"><?= htmlspecialchars($row['email']) ?
    |   ></small>
293 |                                 </td>
294 |                                 <td><?= htmlspecialchars($row['telepon']) ?></td>
295 |                                 <td><?= htmlspecialchars($row['alamat']) ?></td>
296 |                                 <td>
297 |                                     <a href="index.php?edit=<?= $row['id'] ?>" class="btn btn-edit">Edit
    |   </a>
298 |                                     <a href="index.php?hapus=<?= $row['id'] ?>" class="btn btn-hapus" on
    |   click="return confirm('Hapus pelanggan ini?')">Hapus</a>
299 |                                 </td>
300 |                             </tr>
301 |                         <?php endforeach; ?>
302 |                     <?php endif; ?>
303 |                 </tbody>
304 |             </table>
305 |         </div>
306 |     </div>
307 | </div>
308 |
309 | </body>
310 | </html>
```

5.2 Luaran 2: Seluruh Source Code Aplikasi 3-Layer Architecture (3 Folder Layer — 7 File Lengkap)

Pada pendekatan **3-Layer Web Application Architecture**, kode aplikasi dipisahkan secara tegas ke dalam tiga folder sesuai lapisan tanggung jawabnya, yaitu 2-layered-app/presentation/, 2-layered-app/business-logic/, dan 2-layered-app/data-access/. Berikut adalah seluruh source code lengkap dari ke-7 file penyusun aplikasi 3-Layer:

A.1 Presentation Layer — Struktur Halaman Antarmuka (HTML Murni) — Lokasi File: 2-layered-app/presentation/index.html (Total: 108 Baris Lengkap)

File index.html murni menangani struktur tampilan antarmuka pengguna (form input, tombol aksi, kotak pencarian, tabel pelanggan, dan panel monitor respons JSON) tanpa mengandung satu baris pun kode PHP atau query SQL:

Source Code Lengkap: 2-layered-app/presentation/index.html (Bagian 1/3)

```
001 | <!-- ========================================================================
002 |      LAYER 1: PRESENTATION LAYER — presentation/index.html
003 |      Tugas : Menampilkan antarmuka pengguna (Form & Tabel Pelanggan).
004 |              File ini murni HTML (tanpa kode PHP maupun query SQL).
005 |      ======================================================================== -->
006 | <!DOCTYPE html>
007 | <html lang="id">
008 | <head>
009 |     <meta charset="UTF-8">
010 |     <title>Customer Management System - 3-Layer Architecture</title>
011 |     <link rel="stylesheet" href="assets/css/style.css">
012 | </head>
013 | <body>
014 |
015 | <div class="header">
016 |     <h1>Customer Management System — Aplikasi 2: 3-Layer Architecture</h1>
017 |     <p>Pemisahan Tegas: Presentation Layer (HTML/JS) &harr; Business Logic Layer (REST API JSON) &harr;
    |   Data Layer (MySQL)</p>
018 | </div>
019 |
020 | <div class="container">
021 |     <div class="info-box">
022 |         <strong>Info Arsitektur:</strong> Halaman ini adalah <strong>Presentation Layer</strong> murni.
    |   Setiap aksi (Tambah, Tampil, Edit, Hapus, Cari, &amp;amp; Validasi) dikirim ke <strong>Business Logic La
    |   yer</strong> menggunakan <strong>REST API &amp;amp; JSON</strong>.
023 |     </div>
024 |
025 |     <!-- Kotak Pesan Error Validasi (dari JSON Business Logic Layer) -->
026 |     <div id="box-error" class="alert alert-error">
027 |         <strong>Validasi Gagal (Respons JSON dari Business Logic Layer):</strong>
028 |         <ul id="list-error" style="margin: 6px 0 0; padding-left: 20px;"></ul>
029 |     </div>
030 |
031 |     <!-- Kotak Pesan Sukses -->
032 |     <div id="box-sukses" class="alert alert-success"></div>
033 |
034 |     <div class="grid">
035 |         <!-- KOLOM KIRI: FORM TAMBAH / EDIT PELANGGAN -->
036 |         <div class="card">
037 |             <h2 id="judul-form">Tambah Pelanggan Baru</h2>
038 |             <form id="form-pelanggan">
039 |                 <input type="hidden" id="id_edit" value="">
040 |
041 |                 <div class="form-group">
042 |                     <label>Nama Pelanggan *</label>
043 |                     <input type="text" id="nama" placeholder="Contoh: Andi Pratama">
044 |                 </div>
045 |
046 |                 <div class="form-group">
047 |                     <label>Email Pelanggan (Unik) *</label>
048 |                     <input type="text" id="email" placeholder="Contoh: andi@email.com">
049 |                 </div>
```

Source Code Lengkap: 2-layered-app/presentation/index.html (Bagian 2/3)

```
050 |
051 |                 <div class="form-group">
052 |                     <label>Nomor Telepon *</label>
053 |                     <input type="text" id="telepon" placeholder="Contoh: 0812-3456-7890">
054 |                 </div>
055 |
056 |                 <div class="form-group">
057 |                     <label>Alamat Lengkap *</label>
058 |                     <textarea id="alamat" rows="3" placeholder="Masukkan alamat lengkap..."></textarea>
059 |                 </div>
060 |
061 |                 <button type="submit" id="tombol-simpan" class="btn btn-utama">+ Tambah Pelanggan</butto
    |   n>
062 |                 <button type="button" id="tombol-batal" class="btn btn-batal" style="display: none;" onc
    |   lick="resetForm()">Batal</button>
063 |             </form>
064 |         </div>
065 |
066 |         <!-- KOLOM KANAN: PENCARIAN & TABEL DATA PELANGGAN -->
067 |         <div class="card">
068 |             <h2>Daftar Pelanggan (Total: <span id="jumlah-data">0</span>)</h2>
069 |
070 |             <!-- Form Pencarian Berdasarkan Nama atau Email -->
071 |             <form id="form-cari" class="search-form">
072 |                 <input type="text" id="input-cari" placeholder="Cari berdasarkan nama atau email...">
073 |                 <button type="submit" class="btn btn-utama">Cari</button>
074 |                 <button type="button" class="btn btn-batal" onclick="resetPencarian()">Reset</button>
075 |             </form>
076 |
077 |             <table>
078 |                 <thead>
079 |                     <tr>
080 |                         <th>ID</th>
081 |                         <th>Nama &amp;amp; Email</th>
082 |                         <th>Telepon</th>
083 |                         <th>Alamat</th>
084 |                         <th style="width: 120px;">Aksi</th>
085 |                     </tr>
086 |                 </thead>
087 |                 <tbody id="isi-tabel">
088 |                     <tr>
089 |                         <td colspan="5" style="text-align: center; padding: 20px;">Memuat data dari REST
    |    API...</td>
090 |                     </tr>
091 |                 </tbody>
092 |             </table>
093 |         </div>
094 |     </div>
095 |
096 |     <!-- PANEL BUKTI KOMUNIKASI REST API & JSON ANTAR LAYER -->
097 |     <div class="json-box">
098 |         <div class="json-box-title">
```

Source Code Lengkap: 2-layered-app/presentation/index.html (Bagian 3/3)

```
099 |             <span>Monitor Komunikasi REST API &amp;amp; JSON (Presentation Layer &harr; Business Logic Layer
    |   )</span>
100 |             <span id="info-endpoint">GET ../business-logic/api/customers.php</span>
101 |         </div>
102 |         <pre id="isi-json">Menunggu respons JSON...</pre>
103 |     </div>
104 | </div>
105 |
106 | <script src="assets/js/app.js"></script>
107 | </body>
108 | </html>
```

A.2 Presentation Layer — Desain Tampilan & Tata Letak (CSS3 Terpisah) — Lokasi File: 2-layered-app/presentation/assets/css/style.css (Total: 214 Baris Lengkap)

File style.css mengatur seluruh desain visual, tata letak kartu, tabel, badge status HTTP, dan responsivitas halaman secara terpisah:

Source Code Lengkap: 2-layered-app/presentation/assets/css/style.css (Bagian 1/5)

```
001 | /* ============================================================================
002 |  * LAYER 1: PRESENTATION LAYER — assets/css/style.css
003 |  * Tugas : Mengatur tampilan visual halaman web pada aplikasi 3-Layer.
004 |  * ============================================================================ */
005 |
006 | * {
007 |     box-sizing: border-box;
008 |     font-family: 'Segoe UI', Arial, sans-serif;
009 | }
010 |
011 | body {
012 |     background: #f4f6f9;
013 |     margin: 0;
014 |     padding: 0;
015 |     color: #1e293b;
016 | }
017 |
018 | .header {
019 |     background: #1e3a8a;
020 |     color: white;
021 |     padding: 18px 30px;
022 | }
023 |
024 | .header h1 {
025 |     margin: 0;
026 |     font-size: 22px;
027 | }
028 |
029 | .header p {
030 |     margin: 4px 0 0;
031 |     font-size: 14px;
032 |     opacity: 0.9;
033 | }
034 |
035 | .container {
036 |     max-width: 1150px;
037 |     margin: 20px auto;
038 |     padding: 0 20px 40px;
039 | }
040 |
041 | .info-box {
042 |     background: #eff6ff;
043 |     border-left: 5px solid #2563eb;
044 |     padding: 12px 16px;
045 |     margin-bottom: 18px;
046 |     border-radius: 6px;
047 |     font-size: 14px;
048 | }
049 |
050 | .alert {
051 |     padding: 12px 16px;
052 |     margin-bottom: 18px;
```

Source Code Lengkap: 2-layered-app/presentation/assets/css/style.css (Bagian 2/5)

```
053 |     border-radius: 6px;
054 |     display: none;
055 | }
056 |
057 | .alert-error {
058 |     background: #fef2f2;
059 |     border-left: 5px solid #dc2626;
060 |     color: #991b1b;
061 | }
062 |
063 | .alert-success {
064 |     background: #f0fdf4;
065 |     border-left: 5px solid #16a34a;
066 |     color: #166534;
067 |     font-weight: bold;
068 | }
069 |
070 | .grid {
071 |     display: grid;
072 |     grid-template-columns: 350px 1fr;
073 |     gap: 20px;
074 |     align-items: start;
075 | }
076 |
077 | @media (max-width: 850px) {
078 |     .grid {
079 |         grid-template-columns: 1fr;
080 |     }
081 | }
082 |
083 | .card {
084 |     background: white;
085 |     border: 1px solid #cbd5e1;
086 |     border-radius: 8px;
087 |     padding: 18px;
088 |     box-shadow: 0 1px 3px rgba(0, 0, 0, 0.05);
089 | }
090 |
091 | .card h2 {
092 |     margin-top: 0;
093 |     font-size: 18px;
094 |     border-bottom: 1px solid #e2e8f0;
095 |     padding-bottom: 10px;
096 | }
097 |
098 | .form-group {
099 |     margin-bottom: 12px;
100 | }
101 |
102 | .form-group label {
103 |     display: block;
104 |     font-weight: 600;
```

Source Code Lengkap: 2-layered-app/presentation/assets/css/style.css (Bagian 3/5)

```
105 |     font-size: 13px;
106 |     margin-bottom: 5px;
107 | }
108 |
109 | .form-group input,
110 | .form-group textarea {
111 |     width: 100%;
112 |     padding: 8px 10px;
113 |     border: 1px solid #cbd5e1;
114 |     border-radius: 5px;
115 |     font-size: 14px;
116 | }
117 |
118 | .btn {
119 |     display: inline-block;
120 |     padding: 8px 14px;
121 |     border: none;
122 |     border-radius: 5px;
123 |     cursor: pointer;
124 |     font-weight: 600;
125 |     font-size: 13px;
126 |     text-decoration: none;
127 | }
128 |
129 | .btn-utama {
130 |     background: #2563eb;
131 |     color: white;
132 | }
133 |
134 | .btn-utama:hover {
135 |     background: #1d4ed8;
136 | }
137 |
138 | .btn-batal {
139 |     background: #e2e8f0;
140 |     color: #334155;
141 | }
142 |
143 | .btn-edit {
144 |     background: #dbeafe;
145 |     color: #1d4ed8;
146 |     padding: 5px 10px;
147 | }
148 |
149 | .btn-hapus {
150 |     background: #fee2e2;
151 |     color: #dc2626;
152 |     padding: 5px 10px;
153 | }
154 |
155 | .search-form {
156 |     display: flex;
```

Source Code Lengkap: 2-layered-app/presentation/assets/css/style.css (Bagian 4/5)

```
157 |     gap: 8px;
158 |     margin-bottom: 14px;
159 | }
160 |
161 | .search-form input {
162 |     flex: 1;
163 |     padding: 8px 10px;
164 |     border: 1px solid #cbd5e1;
165 |     border-radius: 5px;
166 | }
167 |
168 | table {
169 |     width: 100%;
170 |     border-collapse: collapse;
171 |     font-size: 14px;
172 | }
173 |
174 | th, td {
175 |     border-bottom: 1px solid #e2e8f0;
176 |     padding: 10px;
177 |     text-align: left;
178 |     vertical-align: top;
179 | }
180 |
181 | th {
182 |     background: #f8fafc;
183 |     font-size: 12px;
184 |     text-transform: uppercase;
185 |     color: #475569;
186 | }
187 |
188 | .json-box {
189 |     margin-top: 20px;
190 |     background: #0f172a;
191 |     color: #a7f3d0;
192 |     padding: 14px 18px;
193 |     border-radius: 8px;
194 |     font-family: Consolas, monospace;
195 |     font-size: 12px;
196 | }
197 |
198 | .json-box-title {
199 |     color: #93c5fd;
200 |     font-weight: bold;
201 |     margin-bottom: 8px;
202 |     display: flex;
203 |     justify-content: space-between;
204 | }
205 |
206 | .json-box pre {
207 |     margin: 0;
208 |     background: #1e293b;
```

Source Code Lengkap: 2-layered-app/presentation/assets/css/style.css (Bagian 5/5)

```
209 |     padding: 10px;
210 |     border-radius: 5px;
211 |     max-height: 150px;
212 |     overflow: auto;
213 |     white-space: pre-wrap;
214 | }
```

A.3 Presentation Layer — Interaksi Pengguna & Komunikasi Fetch REST API JSON (JavaScript) — Lokasi File: 2-layered-app/presentation/assets/js/app.js (Total: 176 Baris Lengkap)

File app.js menangani event klik tombol, pengiriman request HTTP (GET, POST, PUT, DELETE) secara asinkron menggunakan fetch() ke endpoint REST API backend, serta merender data JSON ke dalam tabel HTML tanpa memuat ulang halaman:

Source Code Lengkap: 2-layered-app/presentation/assets/js/app.js (Bagian 1/4)

```
001 | /**
002 |  * ============================================================================
003 |  * LAYER 1: PRESENTATION LAYER — assets/js/app.js
004 |  * Tugas : Menghubungkan tampilan HTML dengan Backend (Business Logic Layer)
005 |  *         menggunakan REST API (fetch) dan format data JSON.
006 |  * ============================================================================
007 |  */
008 |
009 | const URL_API = '../business-logic/api/customers.php';
010 |
011 | // 1. Fungsi untuk mengambil & menampilkan daftar pelanggan (GET)
012 | async function muatDataPelanggan(kataKunci = '') {
013 |     let url = URL_API;
014 |     if (kataKunci !== '') {
015 |         url += '?q=' + encodeURIComponent(kataKunci);
016 |     }
017 |
018 |     const response = await fetch(url);
019 |     const hasil = await response.json();
020 |
021 |     // Tampilkan log JSON di panel bawah untuk bukti komunikasi REST API
022 |     tampilkanLogJson('GET ' + url + ' (HTTP ' + response.status + ')', hasil);
023 |
024 |     if (!hasil.success) {
025 |         tampilkanError(hasil.errors || [hasil.message]);
026 |     }
027 |
028 |     const tbody = document.getElementById('isi-tabel');
029 |     document.getElementById('jumlah-data').textContent = hasil.data ? hasil.data.length : 0;
030 |
031 |     if (!hasil.data || hasil.data.length === 0) {
032 |         tbody.innerHTML = '<tr><td colspan="5" style="text-align:center; padding:20px;">Data pelanggan t
    |   idak ditemukan.</td></tr>';
033 |         return;
034 |     }
035 |
036 |     tbody.innerHTML = hasil.data.map(row => `
037 |         <tr>
038 |             <td><strong>#${row.id}</strong></td>
039 |             <td>
040 |                 <strong>${ row.nama }</strong><br>
041 |                 <small style="color:#1d4ed8;">${ row.email }</small>
042 |             </td>
043 |             <td>${ row.telepon }</td>
044 |             <td>${ row.alamat }</td>
045 |             <td>
046 |                 <button type="button" class="btn btn-edit" onclick="isiFormEdit(${row.id})">Edit</button
    |   >
047 |                 <button type="button" class="btn btn-hapus" onclick="hapusPelanggan(${row.id}, '${row.na
    |   ma}')">Hapus</button>
048 |             </td>
049 |         </tr>
```

Source Code Lengkap: 2-layered-app/presentation/assets/js/app.js (Bagian 2/4)

```
050 |     `).join('');
051 | }
052 |
053 | // 2. Fungsi saat form Tambah / Edit disubmit (POST atau PUT dengan JSON)
054 | document.getElementById('form-pelanggan').addEventListener('submit', async function (e) {
055 |     e.preventDefault();
056 |     sembunyikanPesan();
057 |
058 |     const idEdit = document.getElementById('id_edit').value;
059 |     const dataKirim = {
060 |         nama: document.getElementById('nama').value,
061 |         email: document.getElementById('email').value,
062 |         telepon: document.getElementById('telepon').value,
063 |         alamat: document.getElementById('alamat').value
064 |     };
065 |
066 |     // Jika id_edit terisi gunakan method PUT (Edit), jika kosong gunakan POST (Tambah)
067 |     const method = idEdit !== '' ? 'PUT' : 'POST';
068 |     const url = idEdit !== '' ? `${URL_API}?id=${idEdit}` : URL_API;
069 |
070 |     const response = await fetch(url, {
071 |         method: method,
072 |         headers: { 'Content-Type': 'application/json' },
073 |         body: JSON.stringify(dataKirim)
074 |     });
075 |
076 |     const hasil = await response.json();
077 |     tampilkanLogJson(`${method} ${url} (HTTP ${response.status})`, hasil);
078 |
079 |     if (hasil.success) {
080 |         tampilkanSukses(hasil.message);
081 |         resetForm();
082 |         muatDataPelanggan(document.getElementById('input-cari').value);
083 |     } else {
084 |         tampilkanError(hasil.errors || [hasil.message]);
085 |     }
086 | });
087 |
088 | // 3. Fungsi mengambil 1 data pelanggan ke dalam form untuk diedit (GET ?id=...)
089 | async function isiFormEdit(id) {
090 |     sembunyikanPesan();
091 |     const url = `${URL_API}?id=${id}`;
092 |     const response = await fetch(url);
093 |     const hasil = await response.json();
094 |
095 |     tampilkanLogJson(`GET ${url} (HTTP ${response.status})`, hasil);
096 |
097 |     if (hasil.success && hasil.data) {
098 |         document.getElementById('id_edit').value = hasil.data.id;
099 |         document.getElementById('nama').value = hasil.data.nama;
100 |         document.getElementById('email').value = hasil.data.email;
101 |         document.getElementById('telepon').value = hasil.data.telepon;
```

Source Code Lengkap: 2-layered-app/presentation/assets/js/app.js (Bagian 3/4)

```
102 |         document.getElementById('alamat').value = hasil.data.alamat;
103 |
104 |         document.getElementById('judul-form').textContent = 'Edit Pelanggan #' + hasil.data.id;
105 |         document.getElementById('tombol-simpan').textContent = 'Simpan Perubahan';
106 |         document.getElementById('tombol-batal').style.display = 'inline-block';
107 |     }
108 | }
109 |
110 | // 4. Fungsi menghapus pelanggan berdasarkan ID (DELETE ?id=...)
111 | async function hapusPelanggan(id, nama) {
112 |     if (!confirm(`Hapus pelanggan "${nama}"?`)) return;
113 |     sembunyikanPesan();
114 |
115 |     const url = `${URL_API}?id=${id}`;
116 |     const response = await fetch(url, { method: 'DELETE' });
117 |     const hasil = await response.json();
118 |
119 |     tampilkanLogJson(`DELETE ${url} (HTTP ${response.status})`, hasil);
120 |
121 |     if (hasil.success) {
122 |         tampilkanSukses(hasil.message);
123 |         resetForm();
124 |         muatDataPelanggan(document.getElementById('input-cari').value);
125 |     } else {
126 |         tampilkanError(hasil.errors || [hasil.message]);
127 |     }
128 | }
129 |
130 | // 5. Fungsi pencarian berdasarkan nama atau email
131 | document.getElementById('form-cari').addEventListener('submit', function (e) {
132 |     e.preventDefault();
133 |     const kataKunci = document.getElementById('input-cari').value.trim();
134 |     muatDataPelanggan(kataKunci);
135 | });
136 |
137 | function resetPencarian() {
138 |     document.getElementById('input-cari').value = '';
139 |     muatDataPelanggan('');
140 | }
141 |
142 | // Fungsi bantu mereset form kembali ke mode Tambah
143 | function resetForm() {
144 |     document.getElementById('form-pelanggan').reset();
145 |     document.getElementById('id_edit').value = '';
146 |     document.getElementById('judul-form').textContent = 'Tambah Pelanggan Baru';
147 |     document.getElementById('tombol-simpan').textContent = '+ Tambah Pelanggan';
148 |     document.getElementById('tombol-batal').style.display = 'none';
149 | }
150 |
151 | // Fungsi bantu menampilkan pesan sukses / error
152 | function tampilkanSukses(pesan) {
153 |     const box = document.getElementById('box-sukses');
```

Source Code Lengkap: 2-layered-app/presentation/assets/js/app.js (Bagian 4/4)

```
154 |     box.textContent = '✔ ' + pesan;
155 |     box.style.display = 'block';
156 | }
157 |
158 | function tampilkanError(daftarError) {
159 |     const box = document.getElementById('box-error');
160 |     const ul = document.getElementById('list-error');
161 |     ul.innerHTML = daftarError.map(err => `<li>${err}</li>`).join('');
162 |     box.style.display = 'block';
163 | }
164 |
165 | function sembunyikanPesan() {
166 |     document.getElementById('box-sukses').style.display = 'none';
167 |     document.getElementById('box-error').style.display = 'none';
168 | }
169 |
170 | function tampilkanLogJson(infoMethod, dataJson) {
171 |     document.getElementById('info-endpoint').textContent = infoMethod;
172 |     document.getElementById('isi-json').textContent = JSON.stringify(dataJson, null, 2);
173 | }
174 |
175 | // Panggil otomatis saat halaman pertama kali dibuka
176 | muatDataPelanggan();
```

B.1 Business Logic Layer — Endpoint REST API & Pertukaran Data JSON — Lokasi File: 2-layered-app/business-logic/api/customers.php (Total: 64 Baris Lengkap)

File api/customers.php berperan sebagai pintu masuk (*REST API Controller*) yang menerima request HTTP dari Presentation Layer, memanggil CustomerService untuk menjalankan proses bisnis, lalu mengembalikan status HTTP (200, 201, 404, 422, 500) beserta payload berformat **JSON**:

Source Code Lengkap: 2-layered-app/business-logic/api/customers.php (Bagian 1/2)

```
001 | <?php
002 | /**
003 |  * ============================================================================
004 |  * LAYER 2: BUSINESS LOGIC LAYER — REST API Endpoint (api/customers.php)
005 |  * Tugas : Menerima request HTTP (GET, POST, PUT, DELETE) dari Presentation
006 |  *          Layer (Frontend), memprosesnya melalui CustomerService, dan
007 |  *          mengirimkan hasil kembali dalam format JSON.
008 |  * ============================================================================
009 |  */
010 |
011 | header('Content-Type: application/json; charset=utf-8');
012 |
013 | require_once dirname(__DIR__) . '/CustomerService.php';
014 |
015 | try {
016 |     $service = new CustomerService();
017 |     $method  = $_SERVER['REQUEST_METHOD'];
018 |
019 |     // Membaca data JSON yang dikirim oleh Frontend (untuk POST dan PUT)
020 |     $rawBody = file_get_contents('php://input');
021 |     $input   = json_decode($rawBody, true);
022 |     if (!is_array($input)) {
023 |         $input = $_POST;
024 |     }
025 |
026 |     $id = isset($_GET['id']) ? (int) $_GET['id'] : 0;
027 |
028 |     if ($method === 'GET') {
029 |         // Jika ada parameter ?id=... ambil 1 pelanggan, jika tidak ambil semua / cari (?q=...)
030 |         if ($id > 0) {
031 |             $hasil = $service->getCustomerById($id);
032 |         } else {
033 |             $keyword = $_GET['q'] ?? '';
034 |             $hasil   = $service->listCustomers($keyword);
035 |         }
036 |     } elseif ($method === 'POST') {
037 |         // Tambah pelanggan baru
038 |         $hasil = $service->createCustomer($input);
039 |     } elseif ($method === 'PUT') {
040 |         // Edit pelanggan berdasarkan ID
041 |         $hasil = $service->updateCustomer($id, $input);
042 |     } elseif ($method === 'DELETE') {
043 |         // Hapus pelanggan berdasarkan ID
044 |         $hasil = $service->deleteCustomer($id);
045 |     } else {
046 |         $hasil = [
047 |             'status_code' => 405,
048 |             'success'     => false,
049 |             'message'     => 'Method HTTP tidak didukung.',
050 |         ];
051 |     }
052 |
```

Source Code Lengkap: 2-layered-app/business-logic/api/customers.php (Bagian 2/2)

```
053 |     // Kirim status HTTP dan output JSON ke Presentation Layer
054 |     http_response_code($hasil['status_code'] ?? 200);
055 |     echo json_encode($hasil, JSON_PRETTY_PRINT | JSON_UNESCAPED_UNICODE);
056 | } catch (Throwable $e) {
057 |     http_response_code(500);
058 |     echo json_encode([
059 |         'status_code' => 500,
060 |         'success'     => false,
061 |         'message'     => $e->getMessage(),
062 |         'errors'      => [$e->getMessage()],
063 |     ], JSON_PRETTY_PRINT | JSON_UNESCAPED_UNICODE);
064 | }
```

B.2 Business Logic Layer — Aturan Bisnis & Validasi Data Pelanggan — Lokasi File: 2-layered-app/business-logic/CustomerService.php (Total: 197 Baris Lengkap)

File CustomerService.php berisi seluruh logika dan aturan bisnis aplikasi: memvalidasi kelengkapan kolom nama/email/telepon/alamat, memeriksa format email, memastikan email belum digunakan oleh pelanggan lain (keunikan email), serta mengoordinasikan operasi CRUD ke Data Layer:

Source Code Lengkap: 2-layered-app/business-logic/CustomerService.php (Bagian 1/4)

```
001 | <?php
002 | /**
003 |  * ============================================================================
004 |  * LAYER 2: BUSINESS LOGIC LAYER — CustomerService.php
005 |  * Tugas : Menangani aturan bisnis & validasi aplikasi Customer Management:
006 |  *         1. Memeriksa kelengkapan input (nama, email, telepon, alamat)
007 |  *         2. Memeriksa format email yang benar
008 |  *         3. Memeriksa keunikan email (meminta bantuan Data Layer)
009 |  *         4. Mengatur proses Tampil, Tambah, Edit, Hapus, dan Pencarian
010 |  *
011 |  * Catatan : Layer ini TIDAK menulis query SQL dan TIDAK membuat kode HTML.
012 |  * ============================================================================
013 |  */
014 |
015 | require_once dirname(__DIR__) . '/data-access/CustomerRepository.php';
016 |
017 | class CustomerService
018 | {
019 |     private $repo;
020 |
021 |     public function __construct()
022 |     {
023 |         $this->repo = new CustomerRepository();
024 |     }
025 |
026 |     // 1. Ambil daftar pelanggan (Semua atau hasil Pencarian Nama/Email)
027 |     public function listCustomers($keyword = '')
028 |     {
029 |         $keyword = trim((string) $keyword);
030 |         $data = ($keyword !== '')
031 |             ? $this->repo->search($keyword)
032 |             : $this->repo->findAll();
033 |
034 |         return [
035 |             'status_code' => 200,
036 |             'success'     => true,
037 |             'message'     => 'Data pelanggan berhasil diambil.',
038 |             'meta'        => [
039 |                 'total_all'      => $this->repo->countAll(),
040 |                 'total_filtered' => count($data),
041 |                 'keyword'        => $keyword,
042 |             ],
043 |             'data'        => $data,
044 |         ];
045 |     }
046 |
047 |     // 2. Ambil 1 data pelanggan berdasarkan ID
048 |     public function getCustomerById($id)
049 |     {
050 |         $pelanggan = $this->repo->findById((int) $id);
051 |         if (!$pelanggan) {
052 |             return [
```

Source Code Lengkap: 2-layered-app/business-logic/CustomerService.php (Bagian 2/4)

```
053 |                 'status_code' => 404,
054 |                 'success'     => false,
055 |                 'message'     => "Pelanggan dengan ID #$id tidak ditemukan.",
056 |                 'errors'      => ["Pelanggan dengan ID #$id tidak ditemukan."],
057 |             ];
058 |         }
059 |
060 |         return [
061 |             'status_code' => 200,
062 |             'success'     => true,
063 |             'message'     => 'Detail pelanggan ditemukan.',
064 |             'data'        => $pelanggan,
065 |         ];
066 |     }
067 |
068 |     // 3. Tambah pelanggan baru (dengan Validasi Kelengkapan & Keunikan Email)
069 |     public function createCustomer($input)
070 |     {
071 |         $data   = $this->rapikanInput($input);
072 |         $errors = $this->validasiData($data, null);
073 |
074 |         // Jika validasi gagal, tolak penyimpanan & kirim daftar error
075 |         if (!empty($errors)) {
076 |             return [
077 |                 'status_code' => 422,
078 |                 'success'     => false,
079 |                 'message'     => 'Validasi data pelanggan gagal.',
080 |                 'errors'      => $errors,
081 |             ];
082 |         }
083 |
084 |         // Jika validasi lolos, minta Data Layer menyimpan ke database
085 |         $pelangganBaru = $this->repo->create($data);
086 |
087 |         return [
088 |             'status_code' => 201,
089 |             'success'     => true,
090 |             'message'     => "Pelanggan baru '{$data['nama']}' berhasil ditambahkan!",
091 |             'data'        => $pelangganBaru,
092 |         ];
093 |     }
094 |
095 |     // 4. Edit pelanggan (dengan Validasi Kelengkapan & Keunikan Email)
096 |     public function updateCustomer($id, $input)
097 |     {
098 |         $id = (int) $id;
099 |         $lama = $this->repo->findById($id);
100 |         if (!$lama) {
101 |             return [
102 |                 'status_code' => 404,
103 |                 'success'     => false,
104 |                 'message'     => "Pelanggan dengan ID #$id tidak ditemukan.",
```

Source Code Lengkap: 2-layered-app/business-logic/CustomerService.php (Bagian 3/4)

```
105 |                 'errors'      => ["Data pelanggan tidak ditemukan."],
106 |             ];
107 |         }
108 |
109 |         $data   = $this->rapikanInput($input);
110 |         $errors = $this->validasiData($data, $id);
111 |
112 |         if (!empty($errors)) {
113 |             return [
114 |                 'status_code' => 422,
115 |                 'success'     => false,
116 |                 'message'     => 'Validasi perubahan data gagal.',
117 |                 'errors'      => $errors,
118 |             ];
119 |         }
120 |
121 |         $pelangganUpdate = $this->repo->update($id, $data);
122 |
123 |         return [
124 |             'status_code' => 200,
125 |             'success'     => true,
126 |             'message'     => "Data pelanggan '{$data['nama']}' berhasil diperbarui!",
127 |             'data'        => $pelangganUpdate,
128 |         ];
129 |     }
130 |
131 |     // 5. Hapus pelanggan berdasarkan ID
132 |     public function deleteCustomer($id)
133 |     {
134 |         $id = (int) $id;
135 |         $lama = $this->repo->findById($id);
136 |         if (!$lama) {
137 |             return [
138 |                 'status_code' => 404,
139 |                 'success'     => false,
140 |                 'message'     => "Pelanggan dengan ID #$id tidak ditemukan.",
141 |                 'errors'      => ["Data pelanggan tidak ditemukan."],
142 |             ];
143 |         }
144 |
145 |         $this->repo->delete($id);
146 |
147 |         return [
148 |             'status_code' => 200,
149 |             'success'     => true,
150 |             'message'     => "Data pelanggan '{$lama['nama']}' berhasil dihapus.",
151 |             'data'        => ['deleted_id' => $id],
152 |         ];
153 |     }
154 |
155 |     // Fungsi bantu: merapikan spasi dan huruf kecil pada email
156 |     private function rapikanInput($input)
```

Source Code Lengkap: 2-layered-app/business-logic/CustomerService.php (Bagian 4/4)

```
157 |     {
158 |         return [
159 |             'nama'    => trim((string) ($input['nama'] ?? '')),
160 |             'email'   => strtolower(trim((string) ($input['email'] ?? ''))),
161 |             'telepon' => trim((string) ($input['telepon'] ?? '')),
162 |             'alamat'  => trim((string) ($input['alamat'] ?? '')),
163 |         ];
164 |     }
165 |
166 |     // Fungsi utama aturan bisnis: Validasi Kelengkapan & Keunikan Email
167 |     private function validasiData($data, $excludeId = null)
168 |     {
169 |         $errors = [];
170 |
171 |         // A. Cek kelengkapan seluruh kolom wajib
172 |         if ($data['nama'] === '') {
173 |             $errors[] = 'Nama pelanggan wajib diisi.';
174 |         }
175 |         if ($data['email'] === '') {
176 |             $errors[] = 'Alamat email wajib diisi.';
177 |         } elseif (!filter_var($data['email'], FILTER_VALIDATE_EMAIL)) {
178 |             $errors[] = 'Format alamat email tidak valid.';
179 |         }
180 |         if ($data['telepon'] === '') {
181 |             $errors[] = 'Nomor telepon wajib diisi.';
182 |         }
183 |         if ($data['alamat'] === '') {
184 |             $errors[] = 'Alamat pelanggan wajib diisi.';
185 |         }
186 |
187 |         // B. Cek keunikan email melalui Data Layer
188 |         if ($data['email'] !== '' && filter_var($data['email'], FILTER_VALIDATE_EMAIL)) {
189 |             $duplikat = $this->repo->findByEmail($data['email'], $excludeId);
190 |             if ($duplikat) {
191 |                 $errors[] = "Email '{$data['email']}' sudah digunakan oleh pelanggan '{$duplikat['nama']
    |   }'. Gunakan email yang unik!";
192 |             }
193 |         }
194 |
195 |         return $errors;
196 |     }
197 | }
```

C.1 Data Layer — Konfigurasi Koneksi PDO MySQL & Auto-Seed Database — Lokasi File: 2-layered-app/data-access/Database.php (Total: 49 Baris Lengkap)

File Database.php mengisolasi pembuatan koneksi PDO ke database MySQL customer_management serta secara otomatis menjalankan skrip database/customer_management.sql apabila tabel belum tersedia:

Source Code Lengkap: 2-layered-app/data-access/Database.php (Bagian 1/2)

```
001 | <?php
002 | /**
003 |  * ============================================================================
004 |  * LAYER 3: DATA LAYER — Database.php
005 |  * Tugas : Membuka koneksi langsung ke database MySQL (customer_management).
006 |  *         Struktur tabel dan data awal bersumber dari file SQL:
007 |  *         database/customer_management.sql
008 |  * ============================================================================
009 |  */
010 |
011 | class Database
012 | {
013 |     private static $pdo = null;
014 |
015 |     public static function getConnection()
016 |     {
017 |         if (self::$pdo === null) {
018 |             $host     = "localhost";
019 |             $dbname   = "customer_management";
020 |             $username = "root";
021 |             $password = "";
022 |
023 |             $options = [
024 |                 PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
025 |                 PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
026 |             ];
027 |
028 |             try {
029 |                 // Langsung konek ke database customer_management di MySQL
030 |                 self::$pdo = new PDO("mysql:host=$host;dbname=$dbname;charset=utf8mb4", $username, $pass
    |   word, $options);
031 |             } catch (PDOException $e) {
032 |                 // Jika database customer_management belum di-import di MySQL,
033 |                 // otomatis jalankan file database/customer_management.sql
034 |                 $sqlFile = dirname(__DIR__, 2) . "/database/customer_management.sql";
035 |                 if (strpos($e->getMessage(), "1049") !== false && file_exists($sqlFile)) {
036 |                     $pdoSetup = new PDO("mysql:host=$host;charset=utf8mb4", $username, $password, $optio
    |   ns);
037 |                     $pdoSetup->exec(file_get_contents($sqlFile));
038 |                     self::$pdo = new PDO("mysql:host=$host;dbname=$dbname;charset=utf8mb4", $username, $
    |   password, $options);
039 |                 } else {
040 |                     throw new RuntimeException(
041 |                         "Koneksi MySQL gagal. Pastikan MySQL di XAMPP/Laragon sudah dinyalakan. Detail:
    |   " . $e->getMessage()
042 |                     );
043 |                 }
044 |             }
045 |         }
046 |
047 |         return self::$pdo;
048 |     }
```

Source Code Lengkap: 2-layered-app/data-access/Database.php (Bagian 2/2)

```
049 | }
```

C.2 Data Layer — Eksekusi Query SQL Prepared Statement (CRUD & Pencarian) — Lokasi File: 2-layered-app/data-access/CustomerRepository.php (Total: 101 Baris Lengkap)

File CustomerRepository.php bertanggung jawab penuh terhadap seluruh perintah SQL (SELECT, INSERT, UPDATE, DELETE, pencarian nama/email, dan pengecekan email unik) ke tabel pelanggan menggunakan PDO *Prepared Statements*:

Source Code Lengkap: 2-layered-app/data-access/CustomerRepository.php (Bagian 1/3)

```php
<?php
/**
 * ============================================================================
 * LAYER 3: DATA LAYER — CustomerRepository.php
 * Tugas : Menjalankan query SQL (SELECT, INSERT, UPDATE, DELETE, Pencarian)
 *         ke tabel `pelanggan`.
 *         Dipanggil oleh Business Logic Layer (CustomerService.php).
 * ============================================================================
 */

require_once __DIR__ . '/Database.php';

class CustomerRepository
{
    private $db;

    public function __construct()
    {
        $this->db = Database::getConnection();
    }

    // 1. Mengambil seluruh data pelanggan (Fitur Tampil Pelanggan)
    public function findAll()
    {
        $stmt = $this->db->query("SELECT * FROM pelanggan ORDER BY id DESC");
        return $stmt->fetchAll();
    }

    // 2. Mencari pelanggan berdasarkan Nama atau Email (Fitur Pencarian)
    public function search($keyword)
    {
        $stmt = $this->db->prepare("SELECT * FROM pelanggan WHERE nama LIKE ? OR email LIKE ? ORDER BY i
  d DESC");
        $stmt->execute(["%$keyword%", "%$keyword%"]);
        return $stmt->fetchAll();
    }

    // 3. Mengambil 1 pelanggan berdasarkan ID (Untuk Edit / Cek Hapus)
    public function findById($id)
    {
        $stmt = $this->db->prepare("SELECT * FROM pelanggan WHERE id = ?");
        $stmt->execute([(int) $id]);
        $row = $stmt->fetch();
        return $row ?: null;
    }

    // 4. Mencari pelanggan berdasarkan Email (Untuk Validasi Keunikan Email)
    public function findByEmail($email, $excludeId = null)
    {
        if ($excludeId !== null) {
            $stmt = $this->db->prepare("SELECT * FROM pelanggan WHERE LOWER(email) = LOWER(?) AND id !=
  ?");
```

Source Code Lengkap: 2-layered-app/data-access/CustomerRepository.php (Bagian 2/3)

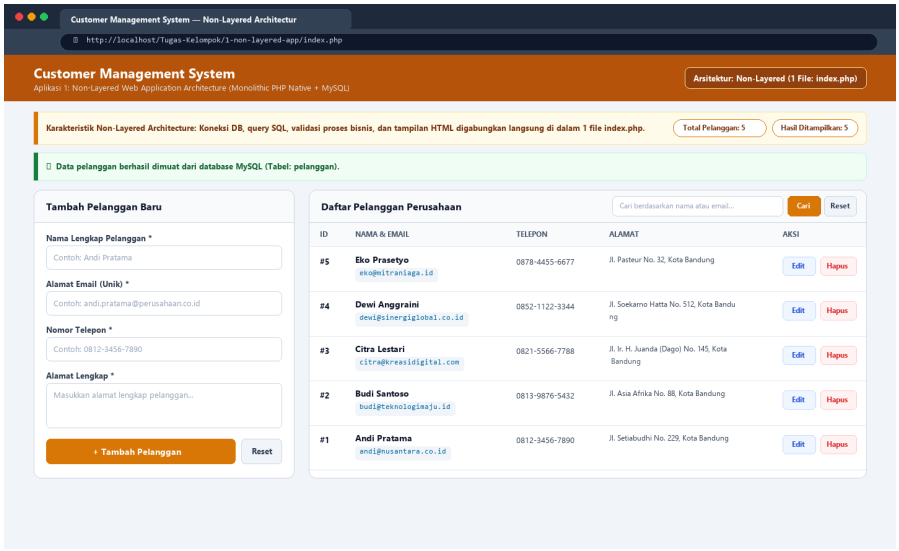
```
051 |             $stmt->execute([$email, (int) $excludeId]);
052 |         } else {
053 |             $stmt = $this->db->prepare("SELECT * FROM pelanggan WHERE LOWER(email) = LOWER(?)");
054 |             $stmt->execute([$email]);
055 |         }
056 |         $row = $stmt->fetch();
057 |         return $row ?: null;
058 |     }
059 |
060 |     // 5. Menyimpan pelanggan baru ke database (Fitur Tambah Pelanggan)
061 |     public function create($data)
062 |     {
063 |         $stmt = $this->db->prepare("INSERT INTO pelanggan (nama, email, telepon, alamat) VALUES (?, ?, ?
    |   , ?)");
064 |         $stmt->execute([
065 |             $data['nama'],
066 |             $data['email'],
067 |             $data['telepon'],
068 |             $data['alamat'],
069 |         ]);
070 |         $idBaru = (int) $this->db->lastInsertId();
071 |         return $this->findById($idBaru);
072 |     }
073 |
074 |     // 6. Mengubah data pelanggan di database (Fitur Edit Pelanggan)
075 |     public function update($id, $data)
076 |     {
077 |         $stmt = $this->db->prepare("UPDATE pelanggan SET nama = ?, email = ?, telepon = ?, alamat = ? WH
    |   ERE id = ?");
078 |         $stmt->execute([
079 |             $data['nama'],
080 |             $data['email'],
081 |             $data['telepon'],
082 |             $data['alamat'],
083 |             (int) $id,
084 |         ]);
085 |         return $this->findById((int) $id);
086 |     }
087 |
088 |     // 7. Menghapus data pelanggan dari database (Fitur Hapus Pelanggan)
089 |     public function delete($id)
090 |     {
091 |         $stmt = $this->db->prepare("DELETE FROM pelanggan WHERE id = ?");
092 |         $stmt->execute([(int) $id]);
093 |         return $stmt->rowCount() > 0;
094 |     }
095 |
096 |     // Menghitung total seluruh pelanggan
097 |     public function countAll()
098 |     {
099 |         return (int) $this->db->query("SELECT COUNT(*) FROM pelanggan")->fetchColumn();
100 |     }
```

Source Code Lengkap: 2-layered-app/data-access/CustomerRepository.php (Bagian 3/3)

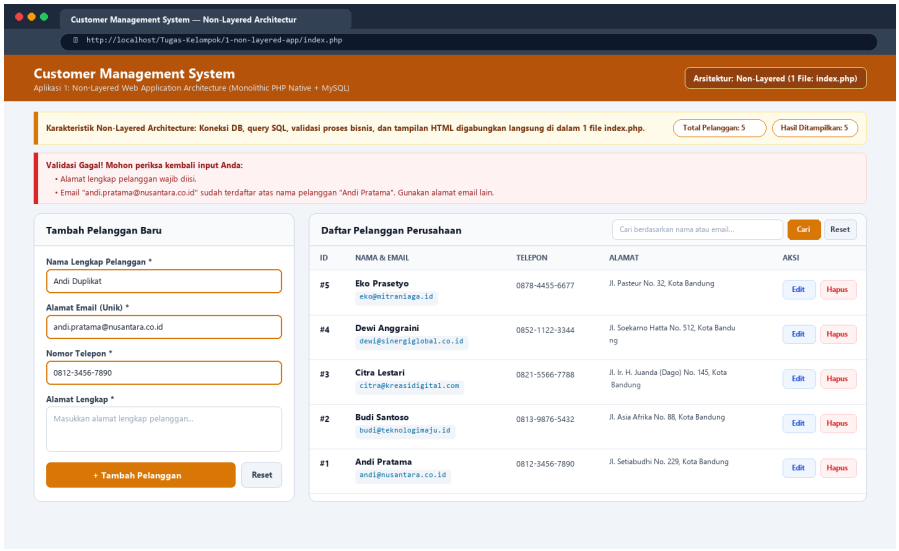
```
101 | }
```

BAB 6. LUARAN 5: SCREENSHOT TAMPILAN WEB KEDUA APLIKASI & ALUR PROSES

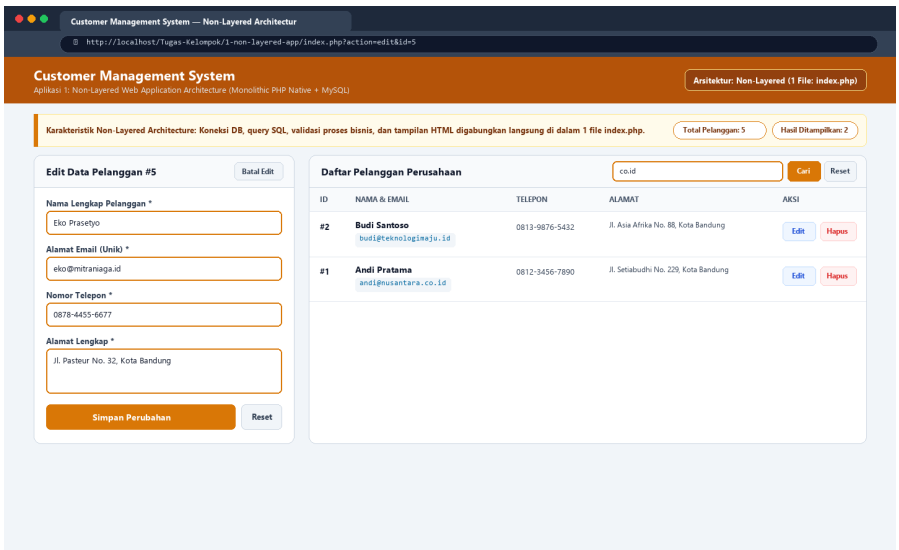
6.1 Screenshot Tampilan Aplikasi 1 (Non-Layered Architecture — Lengkap 3 Skenario Uji)



Gambar 6.1 Tampilan Utama, Tambah Pelanggan, & Tabel Daftar Pelanggan pada Aplikasi 1 (1-non-layered-app/index.php)

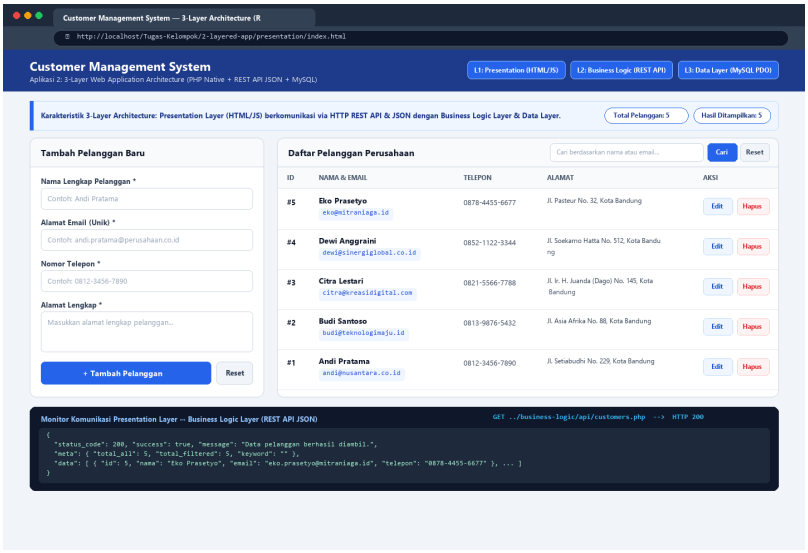


Gambar 6.2 Tampilan Pesan Validasi Kelengkapan Data & Penolakan Email Duplikat pada Aplikasi 1

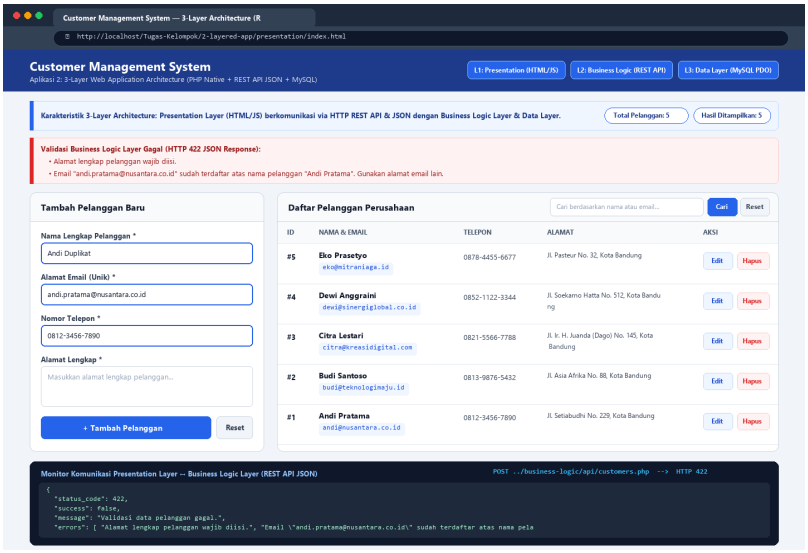


Gambar 6.3 Tampilan Fitur Edit & Pencarian Pelanggan Berdasarkan Nama/Email pada Aplikasi 1

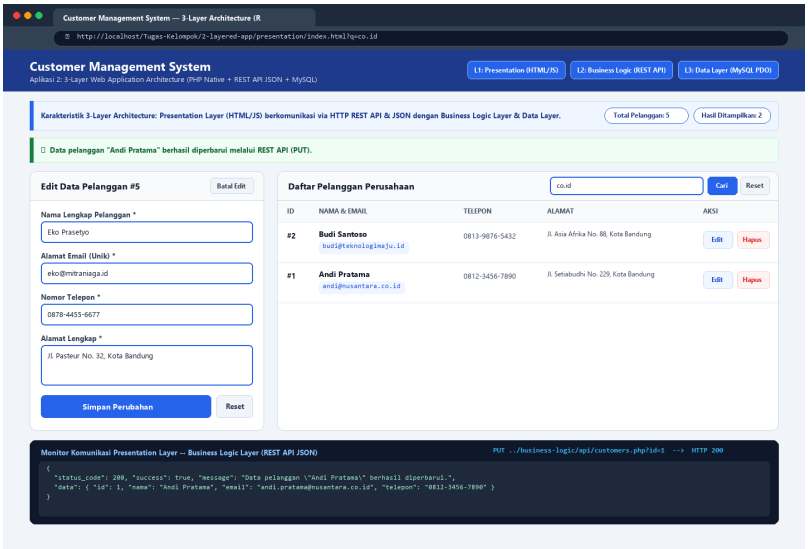
6.2 Screenshot Tampilan Aplikasi 2 (3-Layer Architecture dengan REST API & JSON — Lengkap 3 Skenario Uji)



Gambar 6.4 Tampilan Utama Aplikasi 2 (3-Layer Architecture) beserta Panel Monitor Respons JSON (HTTP 200 OK)



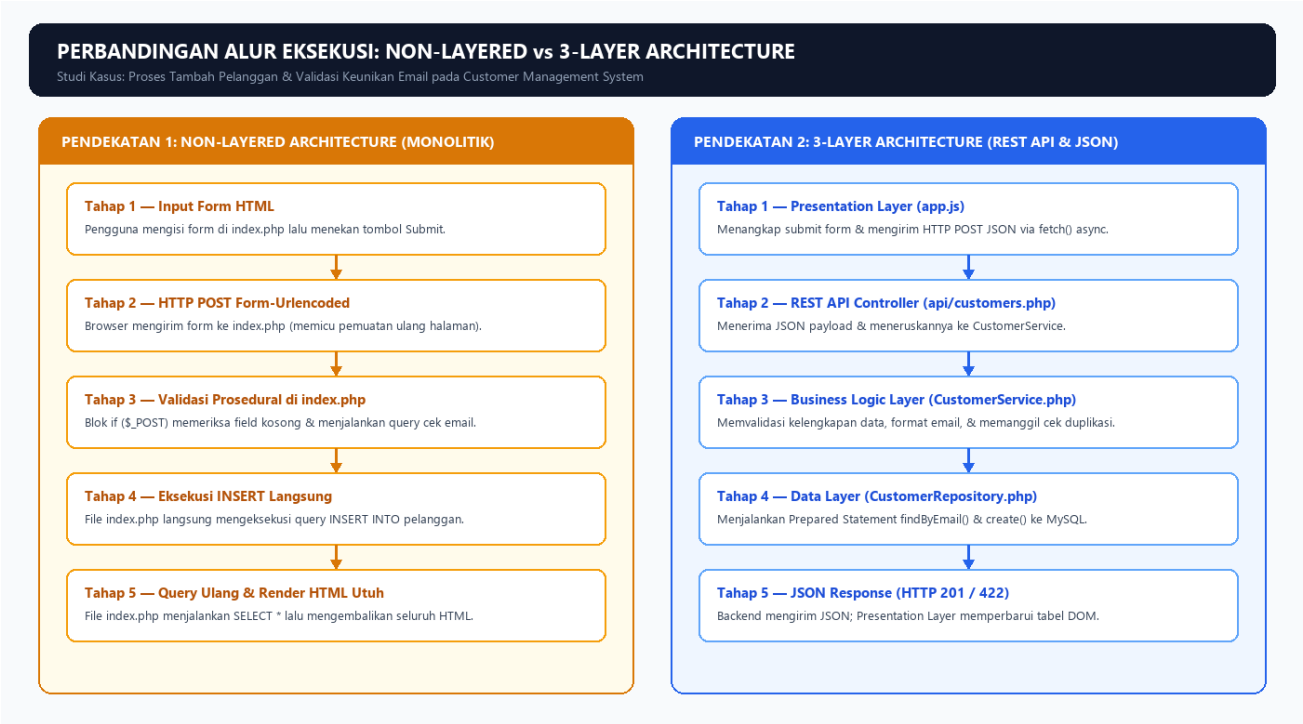
Gambar 6.5 Tampilan Validasi Business Logic Layer (HTTP 422 Unprocessable Entity JSON) pada Aplikasi 2



Gambar 6.6 Tampilan Edit & Pencarian Asinkron via Fetch REST API (?search=...) & Payload JSON pada Aplikasi 2

6.3 Perbandingan Alur Proses Request & Response Kedua Aplikasi

Setelah melihat demo dan tampilan kedua aplikasi, perbedaan cara kerja di balik layar antara pengiriman form tradisional (*Full Page Reload*) pada Non-Layered dan komunikasi asinkron **REST API & JSON** pada 3-Layer dapat diringkas pada diagram berikut:



Gambar 6.7 Diagram Perbandingan Alur Proses: 6 Tahap Non-Layered (Full Page Reload) vs 7 Tahap 3-Layer (REST API & JSON)

BAB 7. TABEL HASIL PENGUJIAN DAN PERBANDINGAN AKHIR

Sesuai arahan dosen pengampu, berikut adalah **Satu Tabel Utama Hasil Pengujian dan Perbandingan** yang merangkum unsur-unsur apa saja yang diuji coba, bagaimana hasilnya pada pendekatan **Tanpa Layer (Non-Layered)** dibandingkan **Menggunakan 3-Layer**, siapa yang lebih unggul pada masing-masing unsur, beserta alasan lengkapnya:

No	Unsur yang Diuji Coba	Tanpa Layer (Non-Layered)	Menggunakan 3-Layer	Lebih Unggul	Alasan Mengapa Lebih Unggul
1	**Struktur Pemisahan File & Folder (Hubungan HTML, CSS, & PHP)**	Belum memiliki pemisahan tanggung jawab layer yang jelas (bisa 1 atau beberapa file bercampur; pada studi kasus ini di 1 file index.php).	Dipisahkan tegas ke 3 folder: presentation/ (HTML/CSS/JS), business-logic/ (PHP API), & data-access/ (SQL).	**3-Layer Architecture**	Kode jauh lebih rapi dan terorganisasi karena setiap file/folder memiliki satu tanggung jawab layer yang tegas.
2	**Kecepatan Pembuatan Awal (Initial Development Speed)**	Sangat cepat dibuat di awal karena kode HTML, PHP, dan SQL bisa langsung ditulis menyatu tanpa menyiapkan REST API.	Membutuhkan waktu awal lebih lama karena harus membuat kontrak JSON, Service, dan Repository di 3 folder.	**Non-Layered Architecture**	Untuk aplikasi yang sangat kecil/sederhana, menulis langsung tanpa pemisahan layer lebih praktis dan cepat.
3	**Pengerjaan secara Individu vs Tim (Teamwork & Kolaborasi)**	Cocok untuk 1 orang (individu), tetapi sulit bagi tim karena kode tampilan, logika, & SQL belum dipisah secara tegas (di sini menyatu di index.php).	Sangat ideal untuk kerja tim: anggota A mengerjakan folder presentation/, B di business-logic/, C di data-access/.	**3-Layer Architecture**	Pembagian tugas kelompok menjadi paralel sesuai layer dan tidak saling menimpa kode (bebas merge conflict).
4	**Kemudahan Penelusuran Kesalahan (Debugging / Error Tracing)**	Sulit ditelusuri saat kode membesar karena harus mencari apakah error berasal dari HTML, validasi, atau SQL di 1 file.	Sangat mudah dilacak: error tampilan cek di presentation/, error validasi di CustomerService, error SQL di Repository.	**3-Layer Architecture**	Sumber bug langsung terisolasi per folder layer sehingga proses perbaikan kesalahan jauh lebih cepat.
5	**Kecepatan & Kenyamanan Interaksi (Pengalaman Pengguna / UX)**	Setiap kali tambah/cari/hapus data, browser harus memuat ulang (reload) seluruh halaman HTML dari awal.	Lebih cepat & responsif karena hanya mengirim/menerima data JSON via fetch() tanpa me-reload seluruh halaman.	**3-Layer Architecture**	Pertukaran data menggunakan REST API & JSON lebih ringan dibandingkan me-render ulang seluruh halaman HTML.
6	**Kemudahan Pemeliharaan Desain (Maintainability UI vs Database)**	Berisiko tinggi: saat mengubah desain tabel HTML/CSS, programmer bisa tidak sengaja merusak kode query SQL di atasnya.	Sangat aman: perubahan desain di index.html / style.css sama sekali tidak menyentuh file PHP maupun query SQL.	**3-Layer Architecture**	Perubahan pada tampilan antarmuka tidak memengaruhi logika bisnis maupun struktur akses database.
7	**Kemudahan Pengembangan Fitur (Scalability / Fitur Bertambah)**	Semakin banyak fitur ditambahkan, file index.php semakin panjang dan rumit (menjadi spaghetti code).	Sangat mudah dikembangkan: penambahan fitur baru cukup menambah fungsi secara modular di masing-masing layer.	**3-Layer Architecture**	Struktur modular menjaga kode tetap bersih meskipun fitur aplikasi terus bertambah besar.
8	**Keamanan & Penggunaan Ulang (Security & Reusability API)**	Query database berada di file tampilan dan tidak bisa dipakai oleh aplikasi Mobile (karena outputnya HTML).	Database terisolasi di Data Layer, dan endpoint REST API JSON (customers.php) bisa langsung dipakai Web & Mobile.	**3-Layer Architecture**	Satu Backend REST API JSON dapat melayani berbagai platform sekaligus (Web maupun Android/iOS).

Tabel 7.1 Tabel Utama Hasil Pengujian dan Perbandingan Non-Layered vs 3-Layer Architecture

BAB 8. JAWABAN 5 PERTANYAAN DISKUSI DAN KESIMPULAN

8.1 Jawaban Lengkap 5 Pertanyaan Diskusi (Halaman 17 PDF)

1. Mana yang lebih mudah dibuat?

Non-Layered Architecture lebih mudah dan cepat dibuat di awal (khususnya jika dikerjakan sendiri untuk aplikasi berskala kecil), karena programmer cukup menulis koneksi database, validasi PHP, query SQL, dan tampilan HTML di dalam satu file index.php tanpa perlu membuat kontrak REST API JSON maupun membagi kode ke dalam beberapa folder.

2. Mana yang lebih mudah diperbaiki ketika terjadi kesalahan (bug)?

3-Layer Architecture jauh lebih mudah diperbaiki dan ditelusuri. Karena adanya pemisahan folder per layer secara tegas, penelusuran kesalahan (*debugging*) langsung terarah: jika terjadi kesalahan tampilan maka diperiksa di folder presentation/, jika aturan validasi keliru diperiksa di CustomerService.php (folder business-logic/), dan jika query SQL bermasalah diperiksa di CustomerRepository.php (folder data-access/).

3. Mana yang lebih mudah dikembangkan ketika fitur bertambah?

3-Layer Architecture jauh lebih mudah dikembangkan. Penambahan fitur baru maupun pembagian tugas di dalam tim dapat dilakukan secara paralel per folder layer tanpa saling merusak kode anggota lain. Selain itu, endpoint REST API JSON (api/customers.php) dapat langsung digunakan ulang oleh aplikasi Mobile (Android/iOS) tanpa menulis ulang logika bisnis.

4. Apa risiko jika seluruh kode ditempatkan dalam satu file?

Risiko utamanya adalah kode menjadi sangat panjang dan tercampur aduk (*spaghetti code*), perubahan sederhana pada desain HTML/CSS berisiko tinggi merusak logika validasi PHP atau query SQL di atasnya, sangat sulit dikerjakan bersama oleh tim karena rawan bentrok (*merge conflict*) pada satu file yang sama, serta sulit diuji secara terpisah.

5. Apakah aplikasi tanpa 3-layer selalu lebih buruk?

Tidak selalu. Untuk aplikasi yang sangat sederhana, skrip satu halaman, prototipe cepat (*MVP*), atau tugas individu berskala kecil, pendekatan tanpa layer justru praktis dan hemat waktu. Namun, untuk aplikasi perusahaan yang dikerjakan secara tim, membutuhkan pemeliharaan jangka panjang, dan terus berkembang, **3-Layer Architecture mutlak lebih unggul**.

8.2 Kesimpulan Akhir

Berdasarkan seluruh analisis teori dan hasil uji coba implementasi studi kasus **Customer Management System**, dapat disimpulkan bahwa **Non-Layered Architecture** hanya unggul dari segi kecepatan pembuatan awal untuk aplikasi kecil yang dikerjakan oleh satu orang. Sebaliknya, pada 7 dari 8 unsur pengujian lainnya—meliputi **kerapian pemisahan file/folder**, **kemudahan kerja sama tim**, **kemudahan penelusuran kesalahan (debugging)**, **kecepatan respons antarmuka tanpa reload**, **keamanan modifikasi desain**, **kemudahan penambahan fitur baru**, serta **reusabilitas REST API JSON**—pendekatan **3-Layer Web Application Architecture** terbukti jauh lebih unggul.